



كلية الحاسبات والذكاء الاصطناعي
FACULTY OF COMPUTERS & ARTIFICIAL INTELLIGENCES



AI Career Coach

A senior project submitted in partial fulfillment of the requirements for the degree of Bachelor of Computers and Artificial Intelligence.

Project Team

1. Ahmed Hamdy Mohamed
2. Zeyad Mohamed Shehata
3. Zeyad Mohamed Mostafa
4. Mostafa Kamel Telab
5. Omar Essam Omar

Under Supervision of

DR. Esraa Abdallah
Eng. Hossam Fakher

Benha, FEB 2026

Table of Contents

1 INTRODUCTION.....	9
1.1 Problem Definition	10
1.2 Importance of this problem	11
1.3 Problem Solution	13
1.4 Project Objectives.....	14
1.5 Related Work.....	15
1.6 System Features	16
1.7 System Requirements	17
1.7.1 Functional Requirements.....	17
1.7.2 Non-Functional Requirements	18
2 SYSTEM ANALYSIS AND DESIGN	21
2.1 Introduction.....	21
2.2 System Analysis	22
2.2.1 Planning	22
2.2.1.1 Project Scope	22
2.2.1.2 Development Approach.....	23
2.2.1.3 Project Timeline Scheduling.....	25
2.2.1.4 Gantt chart	28
2.2.1.5 PERT chart	29
2.2.1.6 Network Diagram	30
2.2.1.7 Cost Estimation.....	32
2.2.1.8 Risk Assessment and Mitigation	35
2.3 SYSTEM DESIGN	37
2.3.1 Stakeholders	37
2.3.2 Block Diagram	38
2.3.3 Use Case	39

2.3.4 Activity Diagrams.....	43
2.3.4.1 Register	43
2.3.4.2 Admin Login.....	44
2.3.4.3 Upload CV	45
2.3.4.4 system	46
2.3.4.5 Update CV	47
2.3.5 State Diagram.....	48
2.3.5.1 Login State Diagram.....	48
2.3.5.2 Register State Diagram	49
2.3.5.3 Model of AI career coach	50
2.3.6 Class Diagram	51
2.3.7 Sequence Diagram	52
2.3.7.1 Login	52
2.3.7.2 Upload CV	53
2.3.7.3 System.....	54
2.3.8 Context Diagram.....	55
2.3.9 Data Flow Diagram	56
2.3.9.1 DFD Level 1.....	56
2.3.9.2 DFD Level 2.....	57
2.3.10 Database Design	58
2.3.10.1 Schema Diagram.....	58
2.3.10.2 ERD Diagram	59
2.3.11 Mapping Diagram	60
2.3.12 METHODOLOGY	61
2.3.12.1 System Units:	61
2.3.12.2 Project Development Methodology (Agile Approach)	62
2.3.12.3 Model Selection Justification and Comparison	63
3 IMPLEMENTATION.....	67

3.1 Introduction.....	67
3.2 System Architecture.....	68
3.3 User Interface Implementation (Website).....	68
3.3.1 CV Neural Analyzer	68
3.3.2 Resume Analysis Completion Interface	69
3.3.3 Skill Gap Detection Interface	70
3.3.4 Learning and Projects Recommendation Interface	71
3.3.5 Job Matching Engine Interface	72
3.4 Backend Implementation	73
3.4.1 Document Processing Pipeline	74
3.4.2 Embedding and Semantic Representation.....	76
3.4.3 Vector Database and Semantic Search	78
3.4.4 Large Language Model Integration	79
3.4.5 AI Query Processing Workflow	81
3.4.6 API Implementation	83

LIST OF FIGURES

Figure 2-1 SDLC.....	24
Figure 2-2 Task Scheduling	28
Figure 2-3 Gantt Chart.....	29
Figure 2-4 Pert Chart.....	30
Figure 2-5 Network Diagram.....	31
Figure 2-6 Block Diagram.....	39
Figure 2-7 Use Case	40
Figure 2-8 Register (Activity Diagram).....	43
Figure 2-9 Admin Login (Activity Diagram)	44
Figure 2-10 Upload CV (Activity Diagram).....	45
Figure 2-11 System (Activity Diagram)	46
Figure 2-12 Update CV (Activity Diagram).....	47
Figure 2-13 Login (State Diagram)	48
Figure 2-14 Register (State Diagram).....	49
Figure 2-15 System (State Diagram)	50
Figure 2-16 Class Diagram.....	51
Figure 2-17 Login (Sequence Diagram).....	52
Figure 2-18 Upload CV (Sequence Diagram).....	53
Figure 2-19 System (Sequence Diagram)	54
Figure 2-20 Context Diagram	55
Figure 2-21 DFD L-1.....	56
Figure 2-22 DFD L-2.....	57
Figure 2-23 Schema Diagram.....	58

Figure 2-24 ERD Diagram	59
Figure 2-25 Mapping	60
Figure 3-1 CV Upload and Parsing Interface	69
Figure 3-2 Resume Analysis Completion Interface	70
Figure 3-3 Skill Gap Detection Interface	71
Figure 3-4 Learning and Projects Recommendation Interface.....	71
Figure 3-5 Job Matching Engine Interface	72
Figure 3-6 Get File Loader.....	75
Figure 3-7 RecursiveCharacterTextSplitter	75
Figure 3-8 embedding backend configuration.....	76
Figure 3-9 embedding generation workflow	77
Figure 3-10 vector database configuration	78
Figure 3-11 semantic retrieval workflow.....	78
Figure 3-12 LLM backend configuration	80
Figure 3-13 response generation workflow	80
Figure 3-14 the query processing workflow	82
Figure 3-15 base API route.....	83
Figure 3-16 upload endpoint.....	84
Figure 3-17 NLP router	84
Figure 3-18 AI answer generation endpoint.....	85
Figure 3-19 job retrieval endpoint	85

LIST OF Table

Table 2-1 User Account (Use Case).....	41
Table 2-2 Resume Handling and Processing (Use Case).....	41
Table 2-3 Career Analysis and intelligence (Use Case)	42
Table 2-4 Administration and profile Management (Use Case)	42
Table 2-5 Summary Comparison	66

ABSTRACT

The rapid evolution of the job market has increased the complexity of career planning and skill development for job seekers. Many candidates struggle to identify skill gaps, optimize their resumes for Applicant Tracking Systems (ATS), and locate job opportunities that align with their qualifications and experience. To address these challenges, this project presents AI Career Coach, an intelligent career guidance platform powered by artificial intelligence and natural language processing techniques.

The proposed system analyzes user-uploaded resumes to extract essential information such as skills, education, work experience, and achievements. Using advanced matching algorithms, the extracted data is compared with real job descriptions to generate ranked job recommendations based on similarity scores. The platform also performs skill gap analysis by identifying missing or outdated competencies relative to target roles, providing users with clear insights into areas requiring improvement.

In addition, a smart resume optimization module enhances resume content by improving structure, wording, and keyword usage to increase compatibility with ATS systems. The system further recommends personalized learning paths, including online courses, certifications, and practical projects, to help users bridge identified skill gaps. By integrating multiple intelligent modules into a unified platform, AI Career Coach offers a comprehensive solution for career development, supporting informed decision-making and continuous professional growth.

the platform is designed with a modular and scalable architecture that allows seamless integration of additional AI models and data sources in the future. The system emphasizes usability and accessibility by providing a user-friendly interface that delivers clear visualizations and actionable insights.

From a technical perspective, the project demonstrates the practical application of NLP and AI-driven recommendation systems in real-world career guidance scenarios. Overall, AI Career Coach contributes to enhancing employability by transforming static resumes into dynamic career development tools.

Chapter One

1 INTRODUCTION

In today's rapidly evolving job market, securing employment has become increasingly challenging, particularly for fresh graduates and early-career professionals. Modern hiring processes rely heavily on automated screening technologies such as Applicant Tracking Systems (ATS), which evaluate résumés based on structure, keywords, formatting, and overall relevance before they are reviewed by human recruiters. As a result, many qualified candidates are eliminated at early stages of the recruitment pipeline not due to a lack of competence, but because their résumés are not optimized to meet ATS requirements.

The core challenge facing job seekers is not a shortage of talent or ambition, but rather the absence of clear and practical guidance on how to present their skills in alignment with job requirements. Applicants often submit large numbers of applications without receiving feedback or understanding why they are not progressing in the recruitment process. Without visibility into résumé weaknesses, missing skills, or keyword mismatches, job seekers are left to guess how to improve their chances of success, leading to inefficiency, frustration, and repeated rejection.

The AI Career Coach project is designed to address these challenges by delivering an intelligent, fully automated system that supports users throughout the career development and job application process. By leveraging advanced artificial intelligence techniques—particularly Natural Language Processing (NLP)—the system analyzes résumés, interprets job descriptions, and generates personalized, actionable insights. These insights help candidates clearly understand their strengths, weaknesses, and areas requiring improvement.

The project aims to build a comprehensive, end-to-end pipeline capable of processing résumés in PDF and DOCX formats with accurate text extraction and structural analysis. It extracts essential information such as skills, professional experience, education, achievements, and relevant keywords using modern NLP models. The system then compares résumés with job descriptions to generate an objective Match Score that reflects how well a candidate aligns with a specific role. In addition, it performs detailed skills gap analysis to identify missing or

underdeveloped competencies and provides personalized learning recommendations that guide users toward targeted courses and resources.

By transforming the traditional job-search experience into a structured, data-driven process, the AI Career Coach enables job seekers to clearly understand what their résumé lacks, how to improve it, and how to align their profiles with employer expectations. Instead of relying on trial and error, users receive targeted insights and professional guidance that help them stand out in a highly competitive job market. With continuous feedback,

intelligent recommendations, and clear direction, the AI Career Coach empowers individuals to strengthen their résumés, develop critical skills, and approach job applications with greater confidence and precision.

1.1 Problem Definition

In the modern job market, job seekers—particularly fresh graduates and early-career professionals—face a significant challenge due to the lack of visibility and feedback in the application process. Employer adoption of automated Applicant Tracking Systems (ATS) has transformed recruitment, but these systems often filter out qualified candidates before a human recruiter ever reviews their application. ATS software scans résumés for specific keywords, formatting, and structured data, meaning that even skilled applicants may be excluded simply because their résumé lacked the exact terms or design required by the system.

This leads to many résumés being discarded without transparent or useful feedback, making the job search feel like a “black hole” where applications disappear with no explanation, leaving candidates uncertain about how to improve future submissions.

Many candidates are unaware of how to optimize their résumés for ATS compatibility, including how to structure content and select appropriate industry-specific keywords that align with automated parsing requirements. Without this guidance, applicants may inadvertently diminish their chances of passing initial screenings, despite having the necessary qualifications.

In addition to struggling with résumé optimization, job seekers often find the application process inefficient because they receive little to no feedback on why their applications were unsuccessful.

The absence of clear insights into performance or rejection reasons prevents candidates from learning and improving over time, contributing to repeated unsuccessful job searches and wasted effort. Moreover, job seekers typically lack visibility into the specific skills they are missing relative to their target roles.

Without data-driven insights into these gaps, they cannot take targeted actions—such as upskilling or training—to enhance their employability. Finally, most traditional career resources provide generic advice rather than personalized recommendations tailored to an individual's unique skills, experience, and career objectives.

This lack of customized support leaves candidates without a strategic roadmap to achieve their employment goals and adapt to evolving market demands.

The AI Career Coach project addresses these challenges by implementing a fully automated, intelligent system capable of analyzing résumés in PDF and DOCX formats, identifying strengths and weaknesses through advanced natural language processing and skills gap analysis, and matching résumés with job descriptions to generate an objective suitability score (Match Score).

The system also offers personalized learning and improvement recommendations, guiding users toward concrete steps for enhancing their skills and optimizing their applications. By integrating these capabilities, the solution empowers job seekers to better understand how ATS evaluates their résumés,

recognize which skills are missing for their targeted roles, and receive tailored advice to increase their competitiveness in the job market. This approach not only helps candidates optimize their résumés for automated screenings but also supports meaningful career development, making the application process more transparent, actionable, and ultimately more successful.

1.2 Importance of this problem

The significance of addressing the challenges faced by job seekers in the current employment landscape is substantial. With the growing reliance of organizations on automated screening systems such as Applicant Tracking Systems (ATS) to manage large volumes of job applications, many candidates are excluded from the recruitment process not because

they lack the required qualifications, but because their résumés are poorly optimized or missing essential keywords.

This situation creates a systemic barrier that limits access to employment opportunities for many capable and qualified individuals.

Providing clear guidance to improve employability is therefore critical. Job seekers often struggle to present their skills, experiences, and achievements effectively in a way that aligns with job requirements and automated screening mechanisms. Without structured and informed support, candidates may fail to highlight their strengths, significantly reducing their chances of being shortlisted for interviews.

Another major concern is the time and effort wasted during the job application process. Submitting numerous applications without understanding the reasons for rejection leads to frustration and inefficiency. When candidates receive actionable insights and meaningful feedback, they can focus their efforts on suitable roles, refine their applications, and significantly improve their chances of success.

Bridging the skills gap also represents a key aspect of this problem. Many applicants are unaware of the specific skills required for their target roles or the areas in which they fall short. Identifying missing or underdeveloped skills enables individuals to pursue targeted learning and upskilling opportunities, thereby enhancing their employability and long-term career growth.

Supporting career development through personalized recommendations and structured feedback allows job seekers to make informed decisions about their professional paths. Instead of relying on generic advice or trial-and-error strategies, candidates can follow data-driven guidance tailored to their unique profiles and career goals.

By solving this problem, the AI Career Coach project delivers a structured, intelligent, and practical solution that enhances the efficiency and effectiveness of job applications, improves individual career prospects, and contributes to building a more skilled, competitive, and resilient workforce.

1.3 Problem Solution

The AI Career Coach project proposes a comprehensive and intelligent solution to the challenges faced by job seekers in the modern employment landscape. By leveraging advanced techniques in Artificial Intelligence

(AI) and Natural Language Processing (NLP), the system delivers an

automated, personalized, and actionable framework that supports candidates throughout the process of résumé optimization and career development.

The system is designed to accurately process résumés uploaded in PDF and DOCX formats, ensuring reliable text extraction and structural analysis so that all relevant information is captured for further evaluation. Using modern NLP models, it extracts essential details such as professional skills, work experience, educational background, achievements, and relevant keywords, thereby constructing a detailed and structured profile of each candidate's capabilities.

To assess job suitability, the system compares the candidate's résumé with specific job descriptions and generates an objective Match Score. This score reflects the degree of alignment between the candidate's profile and the requirements of the targeted role, helping users clearly understand their competitiveness for a given position. In addition, the system performs a detailed skills gap analysis to identify missing or underdeveloped competencies relative to job requirements, enabling candidates to focus their efforts on areas that require improvement.

Based on the identified skill gaps, the AI Career Coach provides personalized learning recommendations, including relevant courses, projects, and educational resources. These tailored suggestions guide users toward concrete, goal-oriented actions that enhance their employability and support continuous professional growth. Instead of offering generic advice, the system delivers structured, data-driven feedback that clearly explains what needs to be improved and how to achieve it, increasing transparency and effectiveness in career decisionmaking.

By implementing this solution, the AI Career Coach transforms the traditional job application process into a guided and data-driven experience. Candidates are empowered to optimize their résumés,

strategically develop essential skills, and approach job applications with greater confidence and precision. Beyond individual benefits, the solution contributes to a more competent and competitive workforce by ensuring that employers are presented with better-prepared and more suitable candidates.

1.4 Project Objectives

The primary objective of the AI Career Coach project is to design and develop an intelligent, fully automated system that supports job seekers in optimizing their résumés, identifying skill gaps, and receiving personalized career guidance. The focus of Term 1 is to establish the core functionalities of the system and build a solid technical foundation that enables future expansion and advanced features in subsequent terms.

During this phase, the project aims to develop a robust résumé processing module capable of accepting résumés in both PDF and DOCX formats. The system will accurately extract textual and structural information to ensure reliable downstream analysis. Leveraging Natural Language Processing (NLP) techniques, the system will identify and extract essential components such as professional skills, educational background, work experience, and achievements, thereby creating a structured representation of the candidate's profile.

Another key objective is to implement résumé-to-job matching functionality. This component will compare extracted résumé content with job descriptions and generate an objective Match Score that reflects the degree of alignment between a candidate's qualifications and the requirements of a specific role. In parallel, the system will conduct detailed skills gap analysis to identify missing or underdeveloped competencies relative to target job positions, providing users with clear insights into areas requiring improvement.

To support continuous learning and employability enhancement, the project also aims to generate personalized learning recommendations. These recommendations will include relevant courses, projects, and educational resources tailored to each user's identified skill gaps.

Additionally, a user-friendly interface will be developed to present analytical results, Match Scores, skills gaps, and learning recommendations in a clear, intuitive, and actionable manner, ensuring an effective and engaging user experience.

Finally, the project will establish evaluation metrics to assess system performance and reliability. These metrics will include accuracy of text extraction, consistency and relevance of Match Scores, and the effectiveness of learning recommendations. Defining such benchmarks ensures that progress during Term 1 is measurable and that the system meets acceptable performance standards.

By achieving these objectives, Term 1 will deliver a functional prototype of the AI Career Coach that demonstrates its ability to analyze résumés, generate meaningful insights, and guide users toward improved employability. This prototype will serve as a strong foundation for future enhancements, including advanced recommendation systems, automated interview preparation, and multilingual support in later development phases.

1.5 Related Work

The development of intelligent career guidance systems has attracted significant attention in both academic research and industry practice, particularly with the increasing use of Artificial Intelligence (AI) and Natural Language Processing (NLP) to enhance employability and recruitment outcomes. A wide range of studies and commercial platforms have explored automated résumé analysis, skill extraction, and personalized career recommendations as mechanisms to support job seekers in navigating modern hiring processes.

One prominent category of existing solutions focuses on optimizing résumés for Applicant Tracking Systems (ATS). Commercial tools such as JobScan and Resumatch are designed to improve keyword alignment and résumé formatting to increase compatibility with automated screening systems. These tools typically provide feedback related to résumé structure, keyword density, and matching against job descriptions. However, they often offer limited insight into deeper skill deficiencies and generally do not include personalized or structured learning recommendations to address identified gaps.

Another group of solutions consists of AI-powered career guidance platforms such as VMock and CareerLift. These platforms utilize AI and NLP models to analyze résumés, assess professional skills and experience, and suggest areas for improvement. In some cases, they provide benchmarking against industry standards or peer profiles. Despite these capabilities, their recommendation mechanisms are often constrained, focusing primarily on résumé enhancement rather than

offering actionable and targeted learning paths that support long-term skill development and career progression.

Academic research has further highlighted the importance of skills gap analysis in improving employment outcomes. Numerous studies demonstrate that AI-driven systems can compare job descriptions with

candidate profiles to identify missing or underdeveloped skills and propose targeted learning interventions.

These approaches frequently rely on techniques such as embedding-based semantic similarity, named entity recognition for skill identification, and machine learning models for classification and matching. While these methods have shown promising results, many research solutions remain theoretical or lack integration into fully functional, user-oriented systems.

Despite the progress achieved by existing platforms and research efforts, several limitations remain evident. Many current solutions concentrate either on résumé formatting or on skill assessment, with limited integration between the two. Personalized learning recommendations are often generic and insufficiently tailored to individual career goals. Furthermore, few systems provide a comprehensive end-to-end pipeline that spans résumé ingestion, skill extraction, job matching, and actionable career guidance. Robust support for multi-format résumé processing, particularly PDF and DOCX files, is also limited across many existing tools.

The AI Career Coach project aims to build upon these prior approaches by introducing a comprehensive, end-to-end system that integrates résumé analysis, Match Score computation, skills gap identification, and personalized learning recommendations within a unified framework. By combining accurate multi-format text extraction, NLP-based skill analysis, and actionable career guidance, the project addresses the limitations of existing tools and research, delivering a more complete, practical, and user-centered solution for career development and employability enhancement.

1.6 System Features

The AI Career Coach system provides a comprehensive set of intelligent features to support resume analysis and career guidance. It enables users to upload résumés in PDF and DOCX formats, performing automatic text extraction to capture all relevant information accurately. The system then extracts key details including professional skills, work experience, and

educational background, creating a structured and detailed profile of the candidate.

Using this extracted information, the system compares the résumé against selected job descriptions and computes an objective Match Score that reflects the candidate's alignment with the target role. In parallel, it conducts a detailed skills gap analysis to identify missing or

underdeveloped competencies, offering users actionable insights into areas requiring improvement.

Based on this analysis, the system generates personalized learning recommendations, suggesting relevant courses, projects, and other educational resources tailored to the candidate's needs.

All results, including the Match Score, identified skills gaps, and recommended learning paths, are presented through an intuitive and userfriendly dashboard. This interface allows users to easily understand their current status, track improvements, and make informed decisions regarding résumé optimization and career development.

Optional or advanced features include basic progress tracking for multiple résumé uploads to monitor development over time, quick AI-generated summaries delivered through a chatbot interface, and initial job description templates to facilitate testing and validation of the Match Score functionality

1.7 System Requirements

1.7.1 Functional Requirements

These describe what the system must do.

1. User Management

The system shall allow users to create, edit, and delete accounts, allow users to log in using email/password, store and manage user profiles including career goals, preferred roles, and industries

2. CV Analyzer

The system shall allow users to upload resumes in formats (PDF, DOCX, TXT),

The system shall extract education, work experience, skills, and achievements using NLP. , identify weak or missing information in the resume , display extracted results to the user for validation.

3. Job Matching Engine

The system shall fetch real job postings from APIs (Wuzzuf) or internal database ,

return a ranked list of recommended jobs , filter jobs based on location, the user's experience level, and job type.

4. Skill Gap Detector

The system shall identify missing, weak, or outdated skills by comparing the candidate's profile against the requirements of target jobs. It will provide actionable insights into areas that require improvement to enhance employability.

5. Smart Resume Optimizer

The system shall suggest improvements to résumé structure and formatting to enhance readability and ATS compliance. It will propose keyword optimization based on target job descriptions and rewrite poorly phrased sentences using AI-based text enhancement. Finally, the system will generate a fully optimized résumé that the user can download in a ready-to-use format.

6. Learning & Project Recommendation Engine

The system shall recommend online courses tailored to address identified skill gaps. Additionally, it shall suggest real-world GitHub projects to provide hands-on experience aligned with the candidate's development goals.

7. Dashboard & Reporting

The system shall provide an intuitive dashboard summarizing user progress. It will display job match scores, missing skills, and résumé improvement suggestions, enabling users to track their development and make informed decisions regarding their career advancement

1.7.2 Non-Functional Requirements

These describe how the system must operate.

1. Performance Requirements

Resume analysis should complete within 5 seconds. Job matching results should load within 3–7 seconds depending on data size. The system should support at least 5000 concurrent users.

2. Reliability & Availability

The system shall be available at all times and operate effectively even with internet speeds of at least 100 KB/s. Results provided shall be reliable and accurate, leveraging machine learning and AI techniques. User data must be backed up daily to ensure recovery and integrity.

3. Scalability

The system shall scale horizontally to support an increasing number of users and job data. The backend must support load balancing for inference requests to maintain performance under high demand.

4. Security

User résumés and sensitive data, such as passwords, must be encrypted in transit (HTTPS/TLS) and at rest. JWT or OAuth2 protocols shall be used for secure authentication. The system shall comply with data privacy laws, following GDPR-style constraints. Passwords must be strong, with a minimum of eight characters including numbers, uppercase, and lowercase letters.

5. Usability

The user interface must be simple, modern, and mobile-friendly. The system shall provide tooltips, guides, and examples for beginners. Reports and dashboards must be intuitive, visually clear, and easily readable.

6. Maintainability

The system shall follow clean API design principles and a modular architecture. The codebase must be well-documented and developer-friendly. Models shall be updated regularly with new job market data to maintain accuracy and relevance.

7. Interoperability

The system shall integrate with external APIs, including job platforms and learning providers. It must support multiple résumé formats such as PDF and DOCX. Exported résumés must remain compatible with standard ATS systems.

Chapter Two

2 SYSTEM ANALYSIS AND DESIGN

2.1 Introduction

The System Analysis and Design phase represents a critical stage in the development of the AI Career Coach project. This phase involves a comprehensive examination of user requirements, system functionalities, and technical specifications to ensure that the final system effectively meets its intended objectives. By adopting a structured approach, the phase facilitates a clear understanding of the workflow, identification of potential challenges, and design of solutions that are efficient, scalable, and user-friendly.

The primary goal of system analysis is to gather detailed information about both functional and non-functional requirements. This process includes understanding user interactions with the system, the types of résumés and job descriptions to be processed, and the expected outputs such as Match Scores, skill assessments, and personalized learning recommendations. Collecting this information ensures that the system aligns with realworld needs and provides meaningful insights to its users.

System design translates these requirements into a detailed blueprint that guides the development process. This encompasses designing the overall system architecture, defining database structures, creating intuitive user interfaces, and establishing robust data processing pipelines. A well-structured design minimizes implementation errors, ensures seamless integration of system components, and provides a clear roadmap for future enhancements.

In this chapter, the analysis and design process of the AI Career Coach is presented comprehensively. It covers use case identification, data flow modeling, system modules, and architectural considerations, thereby laying

the foundation for the development of a robust, intelligent, and user-centered career guidance platform.

2.2 System Analysis

2.2.1 Planning

This phase focuses on establishing a comprehensive and structured project plan for the AI Career Coach system. By systematically organizing tasks, defining timelines, and allocating resources, the planning process ensures a smooth and efficient development workflow. Effective project planning minimizes potential risks, clarifies individual responsibilities, and provides a clear roadmap for constructing the career guidance platform in a systematic, predictable, and goal-oriented manner.

2.2.1.1 Project Scope

The scope of this project encompasses the design and development of an intelligent AI Career Coach platform that provides automated support for job seekers through résumé analysis, skills assessment, and personalized career guidance. The focus of Term 1 is to implement the core modules of the system, establishing a solid foundation for future enhancements.

The Résumé Upload Module supports both PDF and DOCX formats, performing accurate text extraction and structural analysis to ensure that all relevant information is captured for processing.

The Information Extraction Module leverages advanced Natural Language Processing techniques to identify and extract key data, including professional skills, educational background, work experience, and achievements, creating a structured profile of each candidate.

The Résumé-to-Job Matching Module compares the extracted résumé information with job descriptions and generates an objective Match Score that reflects the degree of alignment between a candidate's qualifications and the requirements of the target role.

The Skills Gap Analysis Module identifies missing or underdeveloped competencies relative to the desired position, providing actionable insights to guide skill improvement.

The Learning Recommendation Engine offers personalized suggestions for courses, projects, and other educational resources that directly address the identified skill gaps, supporting targeted professional development.

The Dashboard and Reporting Module presents analysis results, Match Scores, identified skills gaps, and learning recommendations through an intuitive and userfriendly interface. The system also allows users to download reports in PDF or Excel formats for offline review and record-keeping.

Finally, AI Chatbot Integration enables interactive guidance, providing quick summaries, answering user queries in natural language, and enhancing overall system accessibility and usability.

2.2.1.2 Development Approach

The AI Career Coach project adopts an object-oriented development approach to facilitate the creation of modular, reusable, and maintainable system components. Each module is designed to represent a real-world entity, including résumés, job descriptions, skills, recommendations, dashboards, and chatbot services. This approach ensures a clear mapping between system components and functional requirements while supporting scalability and long-term maintainability.

The system is divided into independent modules, each developed and maintained separately to enhance flexibility and integration. The key modules include:

Resume Upload Module, NLP-Based Information Extraction Module, Match Score Computation Module, Skills Gap Analysis Module, Personalized Learning Recommendation Engine, Dashboard and Reporting Module, AI Chatbot Service, Authentication and Role-Based Access Control

Each module communicates with others through well-defined interfaces, ensuring seamless integration and system robustness.

Development Workflow

The development workflow follows a structured and systematic methodology to ensure efficient implementation and high system quality. The following phases were followed sequentially throughout the project lifecycle:

1. Planning Phase – Defining project goals, scope, timelines, resource allocation, and risk assessment.
2. Data Collection Phase – Gathering résumé samples, job descriptions, and relevant learning resources required for system functionality.
3. Requirements Analysis – Documenting functional and non-functional requirements to ensure alignment with user needs and performance expectations.
4. System Design – Designing the database schema, module architecture, automation pipelines.
5. and user interface/user experience (UI/UX).
6. Implementation – Developing system modules using object-oriented principles to achieve modularity, code reusability, and maintainability.
7. Testing and Evaluation – Conducting functional testing, system testing, and validating NLP outputs to ensure accuracy and reliability.
8. Improvement Phase – Applying enhancements and optimizations based on testing results and feedback.

This structured development methodology ensures the creation of a robust, scalable, and efficient AI Career Coach platform that empowers job seekers to optimize their résumés, identify skill gaps, and enhance employability through personalized and actionable guidance.

As shown in Figure2-1 This diagram illustrates the Software Development Life Cycle, known as SDLC. It shows a step-by-step process for building software, starting with Planning and Analysis. The steps continue through Design, Implementation, Testing, and Deployment. Finally, the cycle ends with Maintenance to make sure the software keeps working well

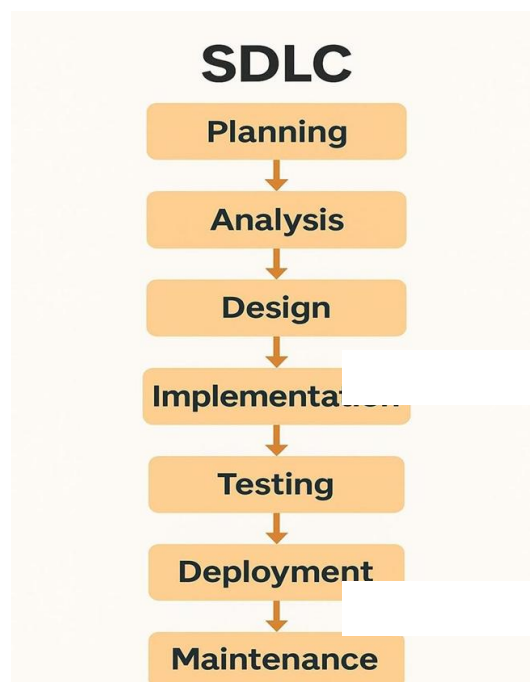


Figure 2-1 SDLC

2.2.1.3 Project Timeline Scheduling

To ensure efficient scheduling of our project development, we have divided the overall development process into two main periods. Within each period, we have further partitioned it into multiple tasks, with each task assigned a specific start date and due date. This approach allows careful time management and ensures that each phase of the project receives adequate attention.

To provide a visual representation of this comprehensive scheduling, Gantt charts have been included to depict the complete system development timeline.

First Development Period

The first development period focuses on project foundation and core model development, including data preparation, system planning, and initial model building. It starts with the Project Scope & Idea, which defines the overall objectives, target users, and functional requirements. This period continues with Data Acquisition and Planning & System Architecture, establishing the necessary resources and designing the system workflow.

During this period, parallel tasks are executed to accelerate progress, including CV Analyzer - NLP Pipeline Development, Job Matching Engine, and Skill Gap Detector +

Visualization. Simultaneously, Smart Resume Optimizer and Learning Path Recommender (Courses) are developed to ensure early integration of intelligent features.

UI and front-end considerations are addressed alongside model development, with UI/UX Design and Front-End Development running in parallel to create a user-friendly interface. By the end of this period.

Second Development Period

The second development period emphasizes system integration, testing, and deployment. Tasks begin with System Integration, combining the front-end and back-end components into a cohesive system. Testing phases, including Unit Tests, Integration Tests, and System Evaluation, are conducted to ensure functionality, performance, and reliability.

Deployment tasks include Cloud Deployment and API Deployment, making the system accessible for users. Concurrently, Documentation (Technical + User Guide) is completed to provide guidance for both developers and end-users.

Performance Monitoring is conducted in the final phase to ensure system stability and efficiency, allowing adjustments if necessary.

Task dependencies and parallel activities are carefully managed to ensure efficient workflow throughout the project. Core model development components, including the CV Analyzer, Job Matching Engine, and Skill Gap Detector, are developed in parallel with UI/UX design and front-end implementation to accelerate system readiness. Deployment activities are scheduled only after successful system integration and testing, while documentation and performance monitoring are carried out alongside final testing to ensure system accuracy, stability, and completeness.

This scheduling ensures structured progress, efficient resource allocation, and timely completion of project deliverables, paving the way for a successful project outcome.

Additionally, risk management considerations were incorporated into the project timeline to mitigate potential delays related to data availability, model performance, and system integration challenges. Buffer time was allocated within critical tasks to handle unforeseen technical issues, particularly during model training and deployment phases. Regular progress reviews and milestone evaluations were scheduled to track task completion and ensure alignment with project objectives.

The use of Gantt charts as a scheduling tool supports transparency, allowing stakeholders to clearly understand task durations, dependencies, and parallel activities throughout the project lifecycle. This structured scheduling approach aligns with standard project management best practices and enhances coordination among development, testing, and documentation activities. Overall, the timeline ensures a balanced workflow that supports quality assurance while maintaining adherence to the planned delivery schedule.

Moreover, milestone-based tracking is applied throughout both development periods to monitor progress and identify potential delays at an early stage. Regular reviews are scheduled at the end of each major milestone to evaluate task completion, resource utilization, and risk factors. This allows corrective actions to be taken promptly without affecting the overall timeline.

Risk management strategies are also integrated into the scheduling plan, where high-risk tasks such as model integration and cloud deployment are allocated buffer time. Agile principles are partially adopted, enabling iterative improvements and continuous feedback, especially during model

development and UI enhancements. The use of Gantt charts not only supports time visualization but also improves coordination among team members by clearly defining responsibilities and task overlaps. This structured yet flexible timeline ensures that both technical quality and project deadlines are achieved efficiently.

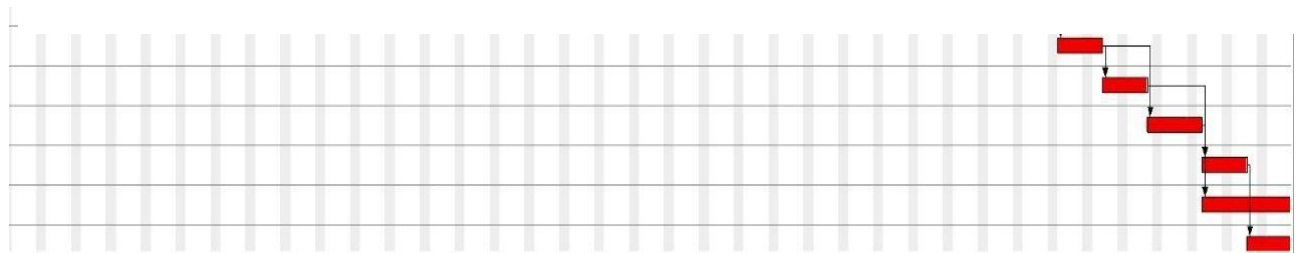
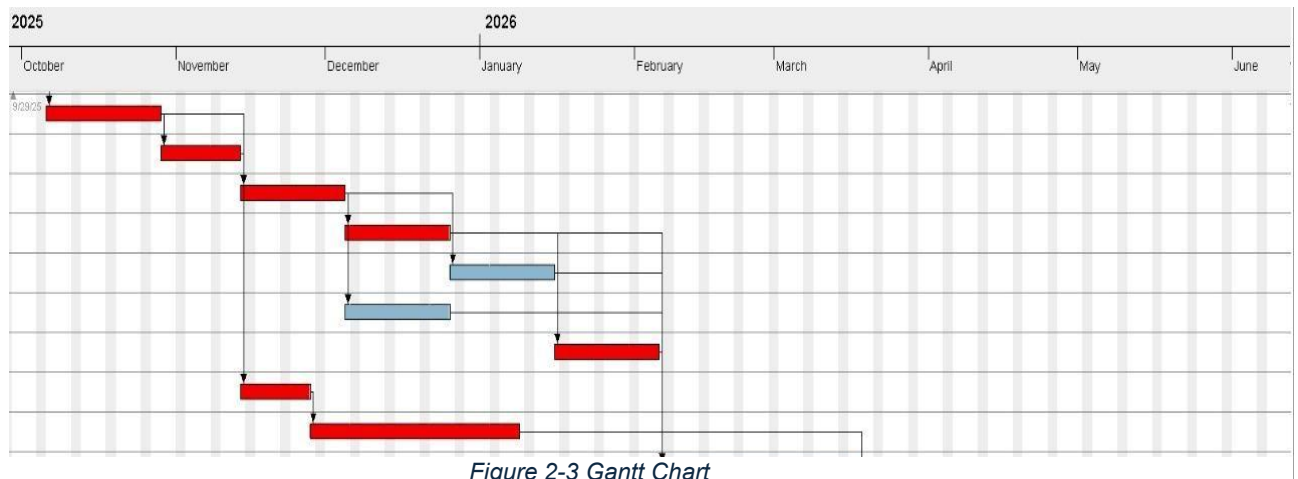
As shown in Figure 2.2 This table displays a timeline for building a software project, listing 18 specific steps from start to finish. It includes tasks like design, development, and testing, showing exactly when each one begins and ends. The project starts in September 2025 and is scheduled to finish in May 2026. This plan helps the team track their progress and manage the time needed for each phase.

Index	Task	Predecessors	Start Date	End Date	Duration
1	Project Scope & Idea	-	25/9/2025	2/10/2025	7
2	Data Acquisition	1	3/10/2025	20/10/2025	17
3	Planning & System Architecture	2	21/10/2025	2/11/2025	12
4	CV Analyzer - NLP Pipeline Development	2,3	3/11/2025	18/11/2025	15
5	Job Matching Engine	4	19/11/2025	4/12/2025	15
6	Skill Gap Detector + Visualization	4,5	5/12/2025	20/12/2025	15
7	Smart Resume Optimizer (ATS + phrasing models)	4	19/11/2025	4/12/2025	15
8	Learning Path Recommender (Courses)	5,6	20/12/2025	4/1/2026	15
9	UI/UX Design	3	4/11/2025	14/11/2025	10
10	Front-End Development	9	31/1/2026	2/3/2026	30
11	Back-End API	5,6,7,8	31/1/2026	28/2/2026	28
12	System Integration (Front + Back)	10,11	3/3/2026	13/3/2026	10
13	Testing - Unit Tests	12	14/3/2026	21/3/2026	7
14	Testing - Integration Tests	13	22/3/2026	29/3/2026	7
15	Testing -System Evaluation	13,14	30/3/2026	6/4/2026	7
16	Deployment (Cloud + API)	15	7/4/2026	12/4/2026	5
17	Documentation (Technical + User Guide)	14,15	6/4/2026	20/4/2026	14
18	Performance Monitoring	16	13/4/2026	13/5/2026	30

Figure 2-2 Task Scheduling

2.2.1.4 Gantt chart

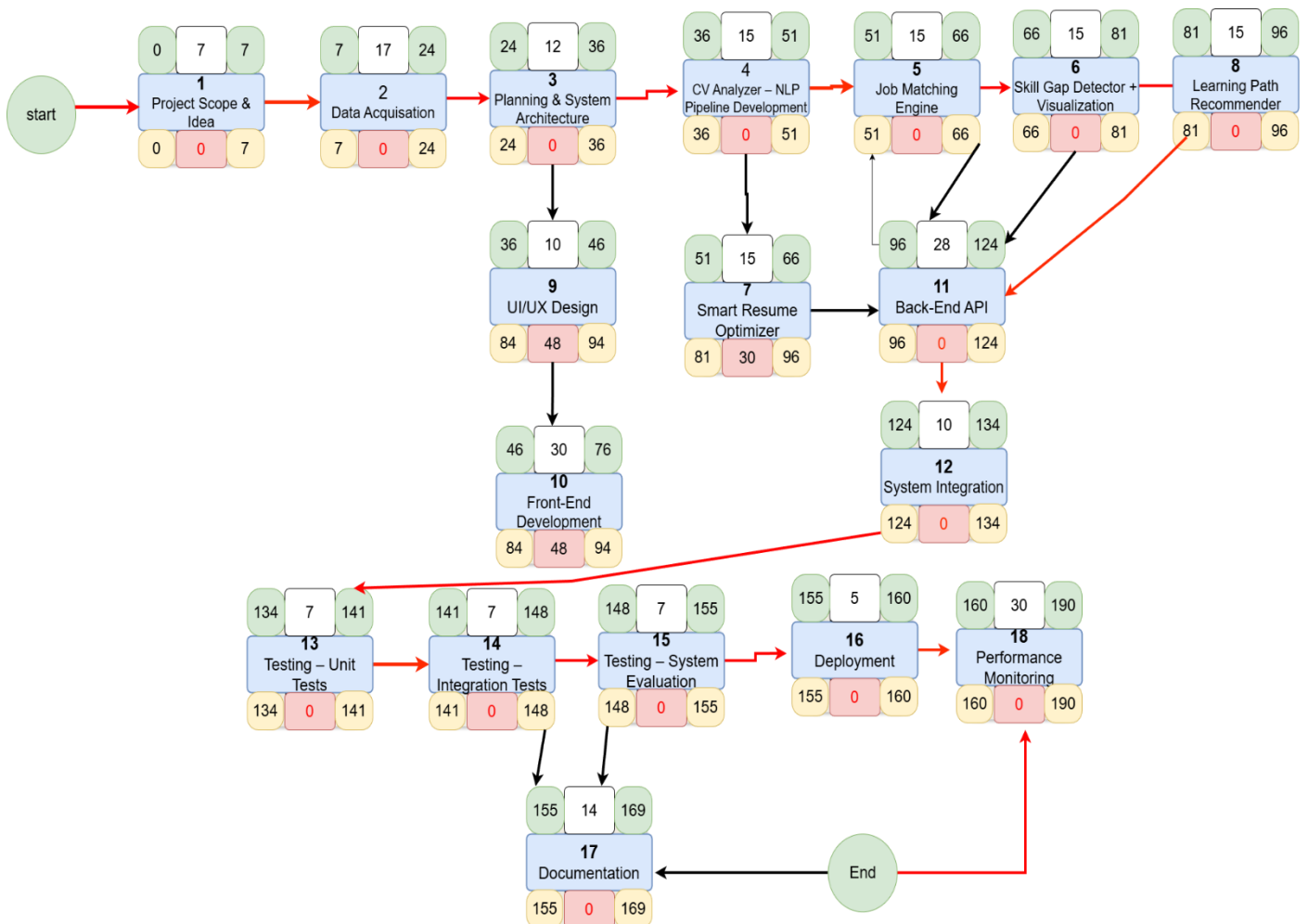
As shown in Figure2-3 This chart visualizes the project timeline using red and blue bars to represent different tasks. It shows the schedule moving step-by-step from October 2025 through to May 2026. The arrows connecting the bars indicate the order of work, showing clearly which tasks must finish before the next ones can begin.



2.2.1.5 PERT chart

As shown in Figure2-4 This diagram shows a project plan from start to end with all tasks and their order. Each box is a task, and the arrows show which task must finish before another starts. The red path is the critical path, meaning any delay there delays the whole project. It helps track time, dependencies, and the total project duration clearly.

Figure 2-4 Pert Chart



2.2.1.6 Network Diagram

As shown in Figure2-5 This diagram outlines the project workflow using circles labeled T1 to T18 to represent different tasks. It traces the path from "Start" to "End," showing exactly how the work progresses step-by-step. The arrows connecting the circles indicate the necessary order, highlighting which tasks must be finished before others can begin.

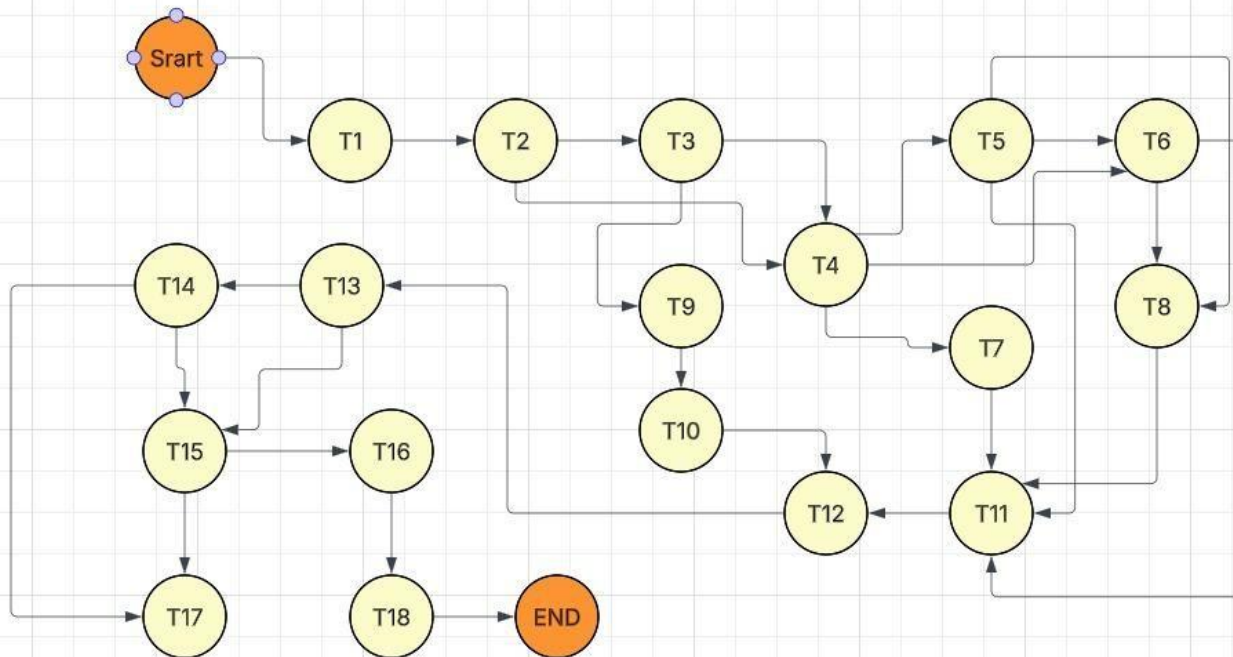


Figure 2-5 Network Diagram

From the PERT chart we could get the following:

- The total number of days needed to complete the development process is 231
- The critical path for the development process is as follows:

S > 1 > 2 > 3 > 9 > 10 > 11 > 12 > 13 > 14 > 15 > 17 > 18 > 19 > 20 > E

The set of tasks that fall into the critical path of the PERT chart are those that have the longest duration and must be completed within their estimated time

to avoid delaying the project. Any delay in these critical path tasks will directly impact the overall project timeline. Tasks that do not fall into the critical path

are those that have some flexibility in their completion time and do not directly impact the overall project duration if they experience a slight delay.

These tasks can be started later or have more flexibility in their scheduling without causing a delay in the project's completion.

2.2.1.7 Cost Estimation

AI Career Coach — Intelligent Career Guidance Platform

This section presents a detailed cost estimation for the development, deployment, and operation of the AI Career Coach platform. The estimation is prepared based on the proposed system architecture, CV analysis pipeline, ATS scoring engine, job matching workflow, selected technologies, and the defined project team structure.

All cost values are adjusted to reflect realistic conditions within the Egyptian market.

Currency Assumption

All costs are presented in Egyptian Pounds (EGP).

International services such as cloud infrastructure, AI processing tool, and third-party APIs are converted from USD to EGP using the following exchange rate: 1 USD = 47.22 EGP

Project Duration and Team Structure

Project Duration: 12 Months , Team Size: 5 Members.

Technology Stack

Frontend: React.js. Backend: Node.js. AI & NLP: Python, spaCy, BERT. Machine Learning Models: Random Forest Regressor. Text Processing & CV Parsing: PyPDF2, pdfplumber, PyMuPDF, OCR.

1. Total Initial Project Cost

The total one-time investment required to design, develop, and deploy the AI Career Coach platform is estimated at **420,000 EGP**.

This cost represents a realistic budget for a graduation-level AI project and a functional MVP within the Egyptian software development market.

1.1 Development and Labor Costs

The total development and labor cost for the project is estimated at **240,000 EGP**.

This amount represents the primary labor expenditure required to complete the AI Career Coach platform within the defined scope and timeline.

The cost calculation is based on a team of **5 members**, with an average monthly compensation of

4,000 EGP per member, over a **12-month development period**.

This labor cost covers frontend and backend development activities, along with AI and machine learning model development. It also includes CV parsing, ATS scoring, and Additionally, the cost accounts for system integration, testing, and validation processes, as well as documentation and deployment tasks necessary to deliver a functional and deployable platform.

1.2 Infrastructure and Software Costs

120,000 EGP

This cost includes all infrastructure and software resources required during both development and deployment phases. It covers cloud servers for development, testing, and production. Managed databases with automated backups are included. File storage for uploaded CV documents is provided. AI model hosting and inference processing (CPU-based) are accounted for. OCR processing support is part of this budget. Monitoring and logging tools are also covered.

1.3 Third-Party Services and AI Processing Fees

40,000 EGP

This budget accounts for the use of external services and resources during development and testing. It includes OCR processing libraries and limited paid services. External datasets and job data sources are covered. Learning platforms and certification recommendation integrations are included.

1.4 Contingency Fund (5%)

20,000 EGP

A contingency reserve is included to mitigate unforeseen risks that may arise during the project. It covers additional cloud resource consumption that could occur due to unexpected usage spikes or extended testing needs.

The fund also accounts for model performance tuning and optimization efforts required if initial results need refinement to meet accuracy or speed targets.

Additionally, it addresses minor scope changes or technical challenges that might emerge and require small adjustments without impacting the main budget.

2. Estimated Ongoing Operational Costs

After deployment, the platform requires continuous operational support to ensure system availability, scalability, and data security. These costs are incurred on a monthly basis to maintain reliable performance and protect user data over time.

Total Estimated Ongoing Operational Cost: 28,000 EGP per month.

2.1 Production Infrastructure

15,000 EGP / Month

Includes live production resources required to process real-time CV uploads and parsing. It executes ATS scoring and job matching requests. Support concurrent users efficiently is covered.

2.2 Third-Party APIs and AI Inference Costs

8,000 EGP / Month

Covers production usage of OCR processing services. Job data APIs are included. Learning resource and recommendation APIs are accounted for. This cost is usage-based and may vary depending on

2.3 Maintenance and Support

5,000 EGP / Month

This cost includes bug fixing and system updates. Model retraining and optimization are covered. Security updates and patches are provided. Limited user support provided by the development team is included.

3. Financial Summary

Total Initial Project Cost: 420,000 EGP. This is a one-time cost spread over a 12-month development period. It covers all expenses required to build and prepare the platform for launch.

Total Ongoing Operational Cost: 336,000 EGP per year. This equals 28,000 EGP per month. The amount supports continuous running of the live system after deployment.

2.2.1.8 Risk Assessment and Mitigation

Risk Assessment:

AI Career Coach — Intelligent Career Guidance Platform

Based on the defined system requirements, architecture, and AI-driven workflows of the AI Career Coach platform, several potential risks have been identified. These risks may affect system performance, accuracy, reliability, and overall project success if not properly managed.

1. Third-Party API Dependency

The platform relies on multiple third-party services, including AI/NLP libraries, OCR tools, and job data sources. Any changes in API availability, pricing models, usage limits, or service discontinuation could negatively impact core functionalities such as CV analysis, ATS scoring, and job matching. Additionally, dependency on external job platforms (e.g., LinkedIn redirection) introduces a risk of broken workflows if URL structures or access policies change.

2. Data Quality & Resume Variability Risk

The accuracy of CV analysis and ATS scoring heavily depends on the quality and structure of uploaded resumes. Variations in resume formats, language styles, scanned documents, or poorly formatted files may lead to incomplete text extraction or incorrect entity recognition.

This risk can result in inaccurate skill extraction, leading to incomplete or misleading candidate profiles.

It may also cause incorrect ATS scores, which can affect the evaluation and ranking of applicants within the system.

Furthermore, the risk can result in weak job matching and recommendations, reducing the overall effectiveness and reliability of the AI Career Coach platform.

3. AI Model Accuracy & Bias Risk

The AI Career Coach system relies on large language models accessed through external APIs to perform key tasks such as resume analysis, skill extraction, job matching, and optimization. Although pre-trained models provide powerful language understanding, they may reflect biases present in their training data and favor certain writing styles or terminology. Because these models were not specifically trained on every possible job domain or resume format, their outputs may occasionally underperform on real-world resumes with diverse structures. Such accuracy and bias risks could affect user trust and the perceived fairness of recommendations if not carefully monitored and evaluated.

4. Performance

The AI Career Coach platform performs a range of computationally intensive operations including document parsing, NLP pipelines, semantic embedding generation, and inference requests to external language model APIs. These processes require significant processing resources and may impact system responsiveness under high load. As the number of users and uploaded resumes increases, the system may encounter latency spikes or fail to meet real-time performance expectations. This risk is particularly apparent when handling large files, simultaneous API calls, or complex semantic matching requests, potentially affecting the overall user experience if not properly monitored and scaled.

5. Data Privacy & Security Risks

The system processes sensitive personal data, including resume content, employment history, and skills and qualifications. Protecting this information is critical to maintaining user trust and complying with privacy regulations. Any failure in implementing proper security controls—such as data encryption, secure authentication, or access control mechanisms—could result in data leaks or unauthorized access. Such breaches may lead to serious privacy concerns and legal consequences for the platform.

6. System Reliability & Availability

Maintaining consistent system availability is critical for user experience. Risks related to cloud service downtime, server failures, or inadequate backup mechanisms could cause service interruptions, data loss, or degraded system performance.

These issues are particularly impactful during peak usage periods, when many users rely on the platform simultaneously. Proper monitoring and redundancy measures are essential to minimize such risks and ensure smooth operation.

Summary

The AI Career Coach platform faces a combination of technical, operational, data-related, and security risks. These risks are common in AI-driven, cloud-based systems and can be mitigated through careful system design, robust infrastructure planning, continuous monitoring, and regular model evaluation.

Identifying these risks early allows the project team to implement appropriate mitigation strategies and ensures the long-term stability and scalability of the platform.

2.3 SYSTEM DESIGN

2.3.1 Stakeholders

Job Seeker (End User): Uploads their résumé (CV) to receive AI-powered analysis of skills, experience, and strengths. Receives a detailed breakdown of skill gaps compared to targeted job roles. Gets an AI-generated Match Score when comparing their CV with real job descriptions. Views customized recommendations to improve ATS compatibility and overall résumé quality.

Accesses a personalized Learning Path that includes courses, certifications, and practical projects. Saves or downloads improved versions of their résumé in multiple formats. Browses job recommendations that align with their current skills and career goals. Gets notified when new job opportunities matching their profile become available. Can track progress and update their profile as they acquire new skills.

Platform Admin (System Owner): Has full administrative privileges over the platform. Manages user accounts, access control, and overall system security. Monitors and updates the job, skills, and learning resources databases. Oversees AI model performance, retraining schedules, and system accuracy. Manages platform content including recommended courses, certifications, and career resources. Generates systemwide reports on user activity, job matching results, and AI performance metrics.

Is responsible for deploying updates, maintaining system stability, and performing periodic maintenance. Ensures data integrity, privacy compliance, and platform scalability.

System Developer / ML Engineer: Designs and builds the platform's architecture, including frontend, backend, and database layers. Develops the NLP pipeline for CV analysis and skill extraction. Implements AI models for job matching, skill gap detection, and résumé optimization. Integrates external

APIs such as job platforms, course providers, and GitHub projects. Ensures the system runs efficiently, securely, and with high accuracy. Works on improving algorithm performance, embedding models, and recommendation quality. Collaborates in system testing, debugging, and optimizing overall user experience. Is responsible for deploying machine learning models into production environments.

2.3.2 Block Diagram

As shown in Figure2-6 This diagram shows how the AI system works, starting with a user logging in and uploading their resume. The system then analyzes the resume to find skills and identifies any missing information or "skill gaps". It generates smart recommendations for job matches and suggests learning paths or courses for the user. Finally, the system saves all this data securely and integrates with external job boards like LinkedIn.

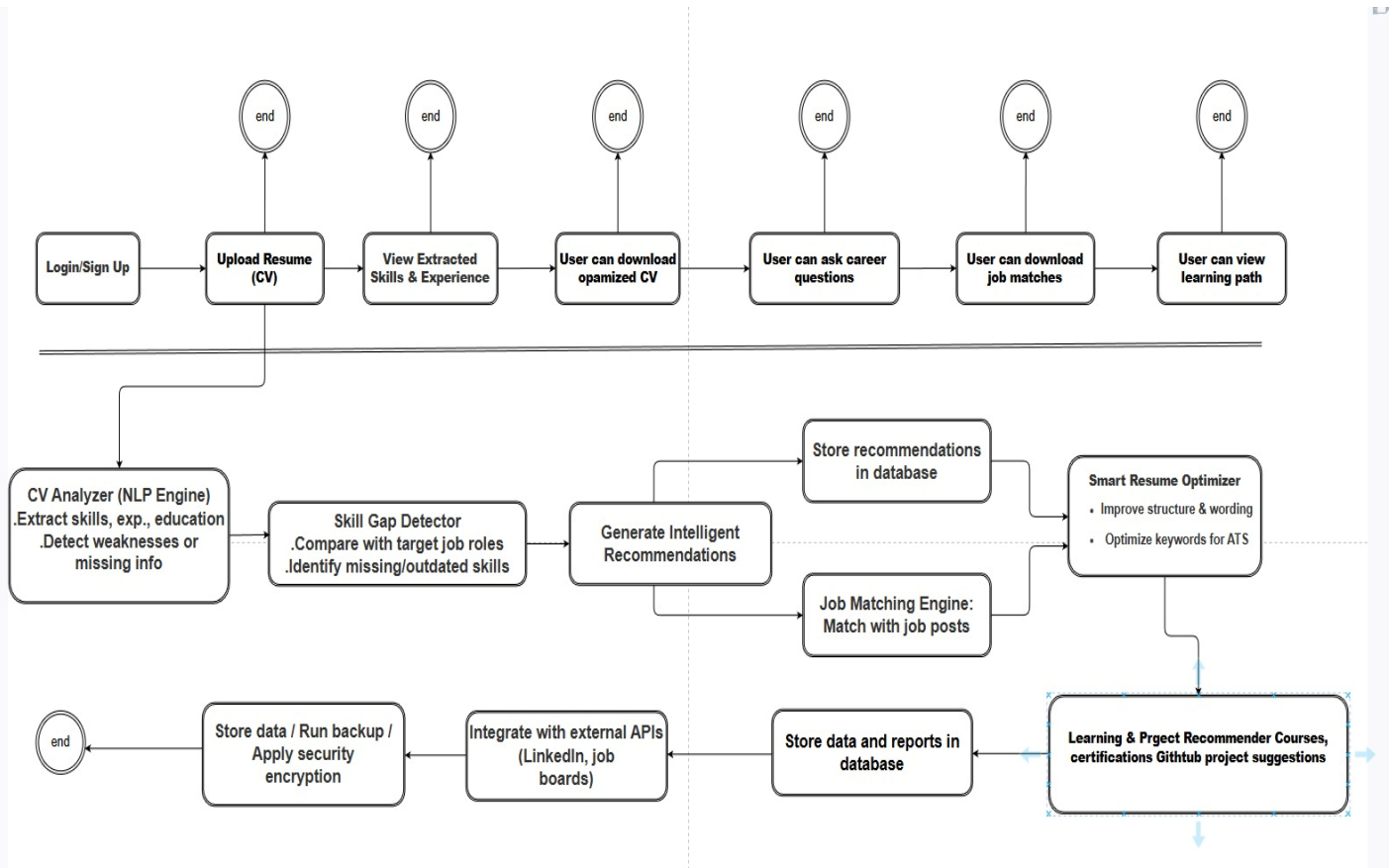


Figure 2-6 Block Diagram

2.3.3 Use Case

As shown in Figure2-7, this diagram presents the main features and data flow of the AI Career Coach (Career AI) system. The user creates an account, logs in, and is verified before accessing the dashboard. User profiles and CV data are securely stored and managed by the system. The uploaded CV is processed by the Smart Resume Optimizer, which works with AI models to improve resume quality. The Job Matching Engine and Skill Gap Detector analyze the user's profile to identify suitable roles and missing skills. Based on this analysis, the system provides personalized learning and project recommendations, and allows the user to download the optimized CV.

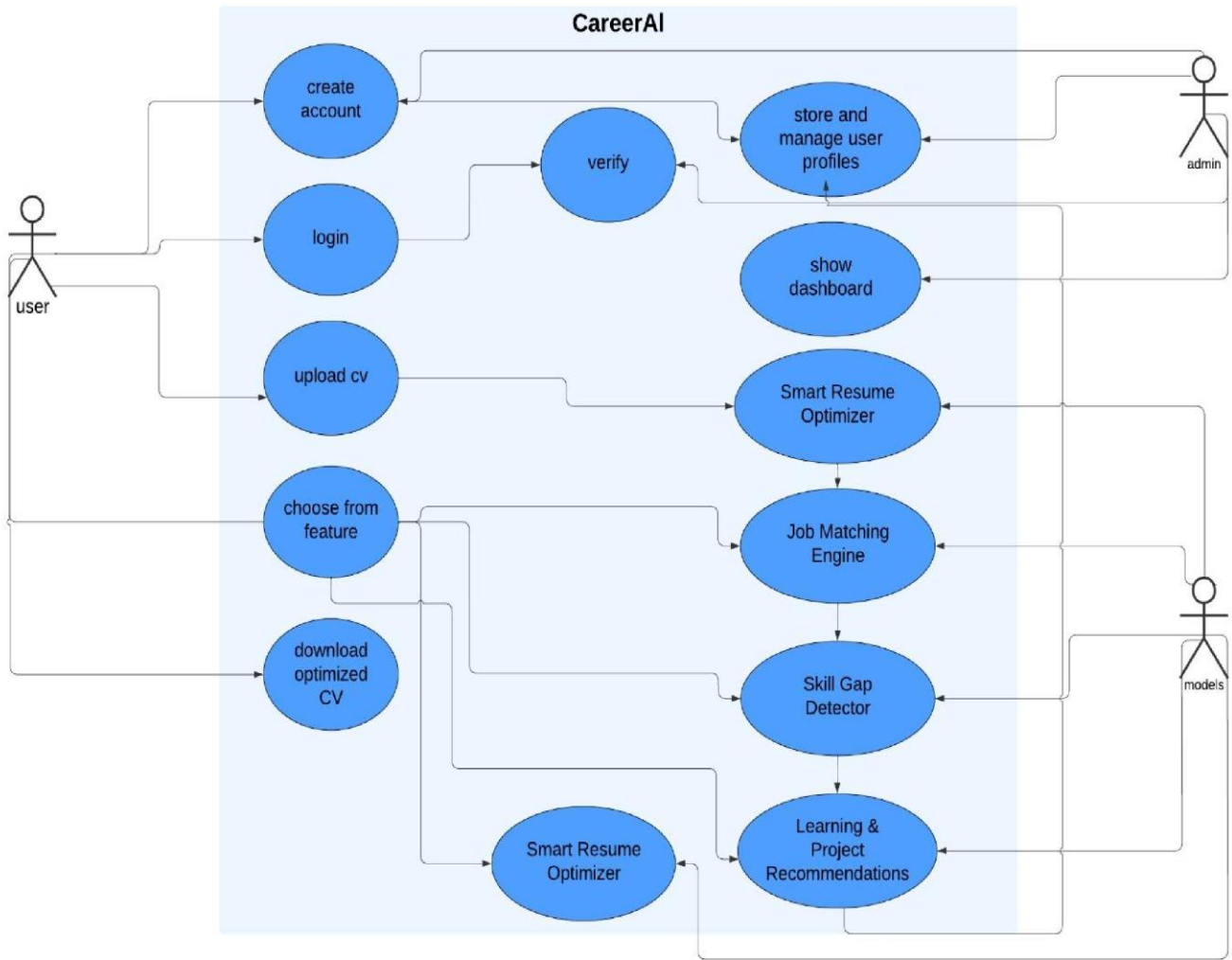


Figure 2-7 Use Case

Table 1 presents the use cases related to user account management within the AI Career Coach platform. It describes the processes that allow users to create accounts, log into the system, and access personalized features securely. The table also highlights the system's role in verifying user credentials during authentication. These use cases ensure secure access control and enable users to view their personalized dashboard after successful login.

Table 2-1 User Account (Use Case)

Use Case Name	Actor	Description	Pre-Conditions	Post-Conditions
Create Account	User	Allows a new user to register in the system by creating an account.	User is not registered.	User account is created and stored.
Login	User	Enables registered users to access the system securely.	User account exists.	User is authenticated and logged in.
Verify User	System	Validates user credentials during login or registration.	Login or registration request is submitted.	User is verified or access is denied.
Show Dashboard	User	Displays user-specific insights and progress information.	User is logged in.	Dashboard is displayed successfully.

Table 2. It focuses on how authenticated users upload their CVs, which are then analyzed and optimized using AI models. The table also describes the generation of an optimized résumé and the ability for users to download it. These use cases support resume improvement and ensure compatibility with Applicant Tracking Systems (ATS).

Table 2-2 Resume Handling and Processing (Use Case)

Use Case Name	Actor	Description	Pre-Conditions	Post-Conditions
Upload CV	User	Allows users to upload their résumé in supported formats.	User is authenticated.	CV is stored and prepared for analysis.
Smart Resume Optimizer	AI Models	Improves résumé structure, wording, and ATS compatibility.	CV is uploaded and analyzed.	Optimized résumé is generated.
Download Optimized CV	User	Allows the user to download the optimized résumé.	Optimization is completed.	Optimized CV is downloaded.

Table 3 describes the use cases related to career analysis and intelligent decision support. It explains how users select specific system features after uploading their resumes. The table also details AI-driven processes such as job matching, skill gap detection, and personalized learning and project recommendations. These use cases aim to provide users with actionable insights to improve employability and career growth.

Table 2-3 Career Analysis and intelligence (Use Case)

Use Case Name	Actor	Description	Pre-Conditions	Post-Conditions
Choose Feature	User	Allows the user to select a desired system feature.	CV is uploaded.	Selected feature is activated.
Job Matching Engine	AI Models	Matches résumé data with job descriptions and calculates a match score.	Resume data is extracted.	Ranked job matches are generated.
Skill Gap Detector	AI Models	Identifies missing or weak skills compared to job requirements.	Job matching is completed.	Skill gaps are identified.
Learning & Project Recommendations	AI Models	Recommends courses and projects based on skill gaps.	Skill gap analysis is completed.	Personalized recommendations are generated.

Table 4 presents the administrative use cases of the system. It focuses on the role of administrators in managing and updating user profile information after authentication. These use cases ensure data consistency, system maintenance, and accurate user information management within the platform

Table 2-4 Administration and profile Management (Use Case)

Use Case Name	Actor	Description	Pre-Conditions	Post-Conditions
Store and Manage User Profiles	Admin	Manages and updates user profile information.	Admin is authenticated.	User profile data is updated.

2.3.4 Activity Diagrams

2.3.4.1 Register

As shown in Figure2-8, this activity diagram describes the user registration and login process in the AI Career Coach system. The process starts when the user opens the system and checks whether they are already registered. If the user is new, they enter registration data; otherwise, they proceed by entering their login credentials. The system then verifies the user information. If the credentials are valid, the login process is completed successfully. Otherwise, an error message is displayed to inform the user of invalid credentials.

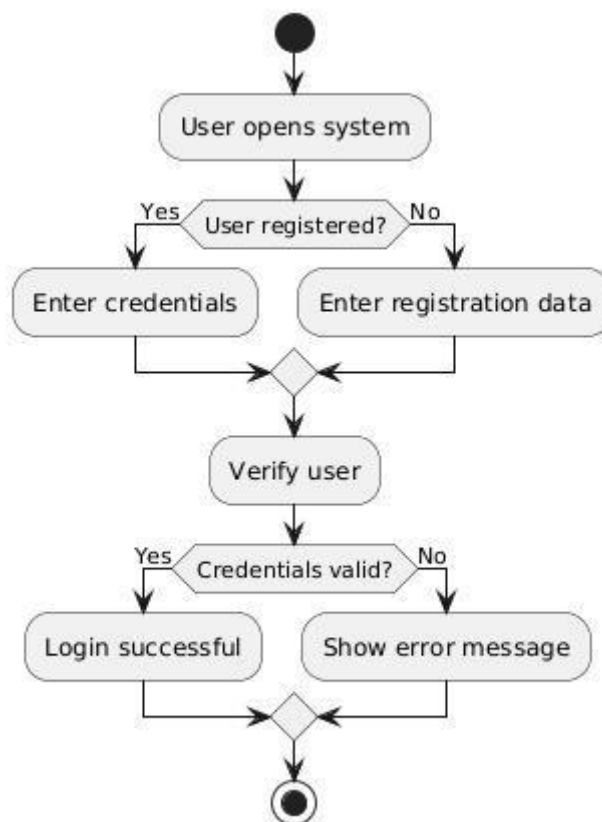


Figure 2-8 Register (Activity Diagram)

2.3.4.2 Admin Login

As shown in Figure 2.9 this activity diagram illustrates the admin profile management process in the AI Career Coach system. The process starts with the admin logging into the system and undergoing authentication. If the admin is successfully authenticated, access to user profiles is granted. The admin can then view, edit, and update user profile information. After making the required changes, the updates are saved to the system. If authentication fails, access is denied and the process is terminated.

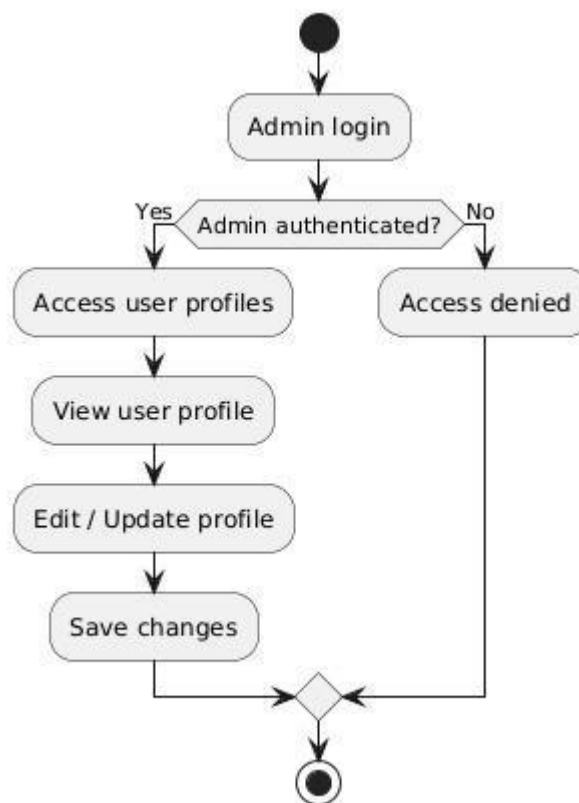


Figure 2-9 Admin Login (Activity Diagram)

2.3.4.3 Upload CV

As shown in Figure2-.10 , this activity diagram illustrates the CV upload process in the AI Career Coach system. The process begins when the user uploads a CV file to the platform. The system then validates the file size and format, allowing only supported types such as PDF, DOC, and DOCX within the specified size limit. If the file meets the validation requirements, it is securely stored in the system. If the file is invalid, the upload is rejected. This process ensures data integrity and proper input for subsequent AI-based analysis

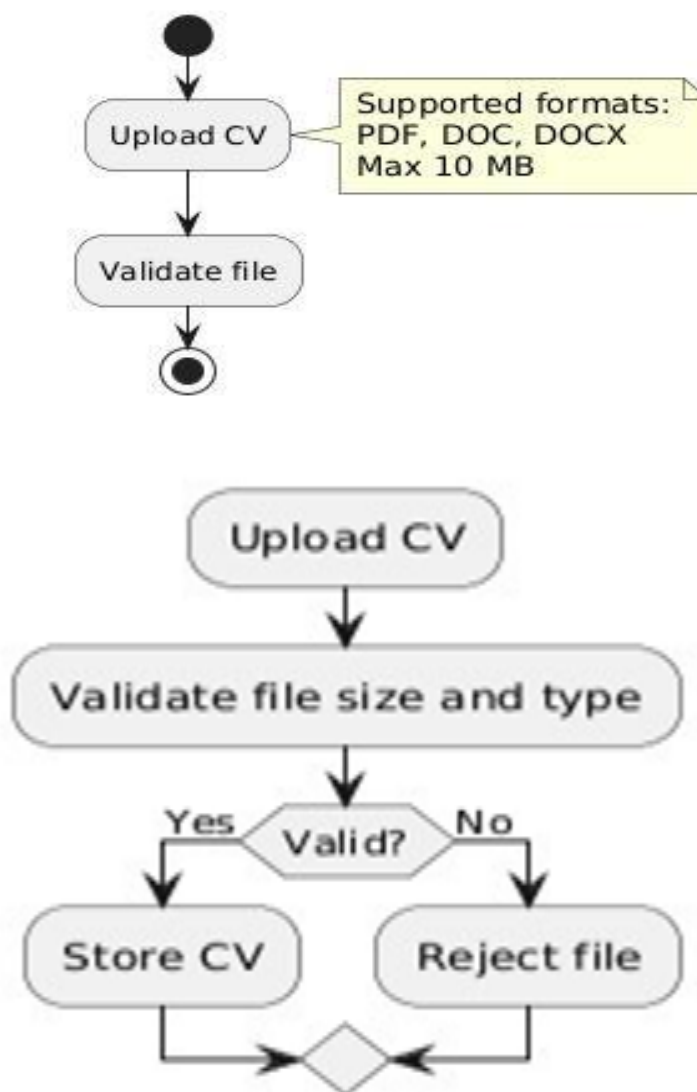


Figure 2-10 Upload CV (Activity Diagram)

2.3.4.4 system

As shown in Figure 2-11, this activity diagram describes the career analysis and recommendation process in the AI Career Coach system. The system first checks whether a CV has been uploaded; if not, the user is prompted to upload one. Once the CV is available, resume data is extracted and processed. The Job Matching Engine analyzes the profile to provide suitable job matches, while the Skill Gap Detector identifies missing skills. Based on this analysis, the Learning and Project Recommendations system generates personalized recommendations. Finally, a comprehensive report is generated for the user.

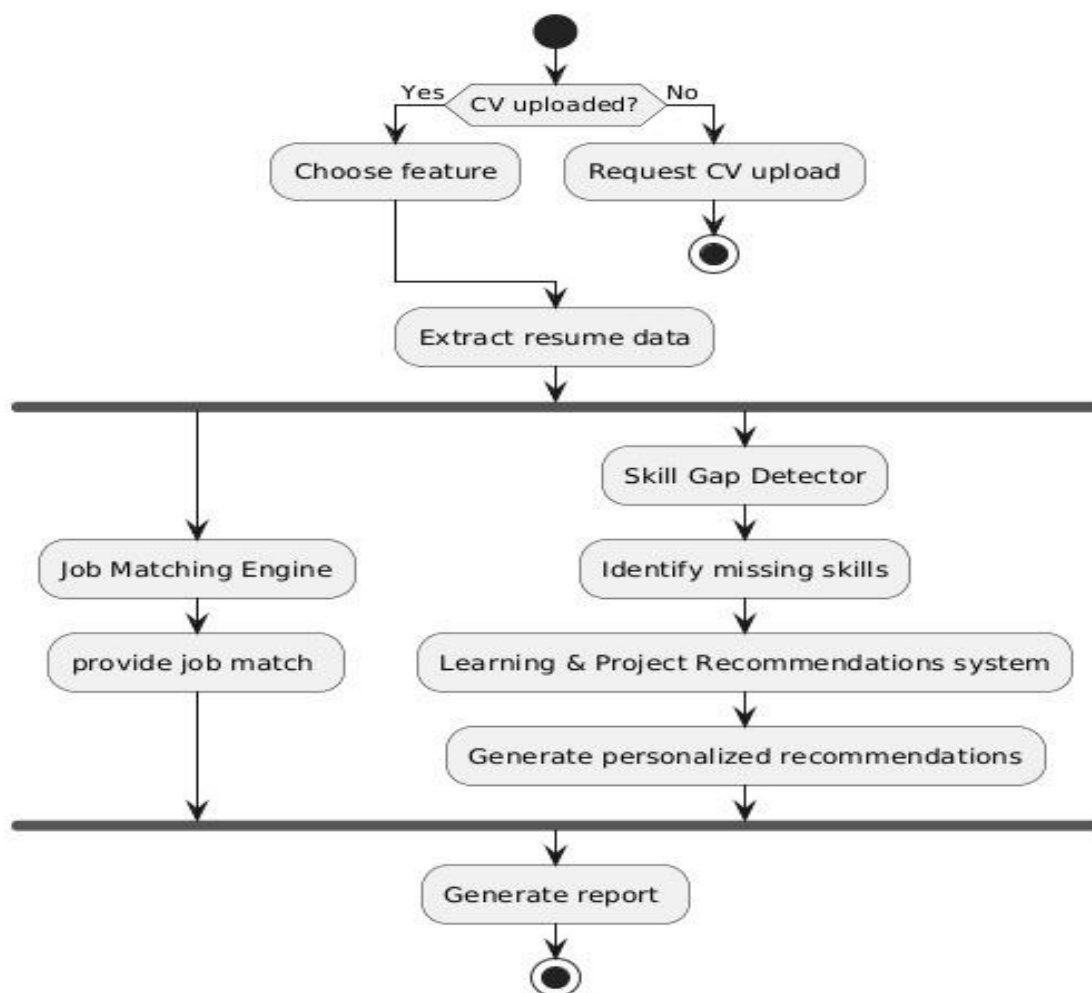


Figure 2-11 System (Activity Diagram)

2.3.4.5 Update CV

As shown in Figure 2-12, this activity diagram illustrates the process of updating user data by uploading a new CV in the AI Career Coach system. The process begins when the user logs into the system and navigates to the profile page. If the user chooses to update the CV, a file is selected and validated based on type and size. When validation is successful, the CV is uploaded to the server and the user profile is updated accordingly. The changes are then saved in the database and the user is notified of a successful upload. If validation fails or the update is canceled, the process ends without making changes.

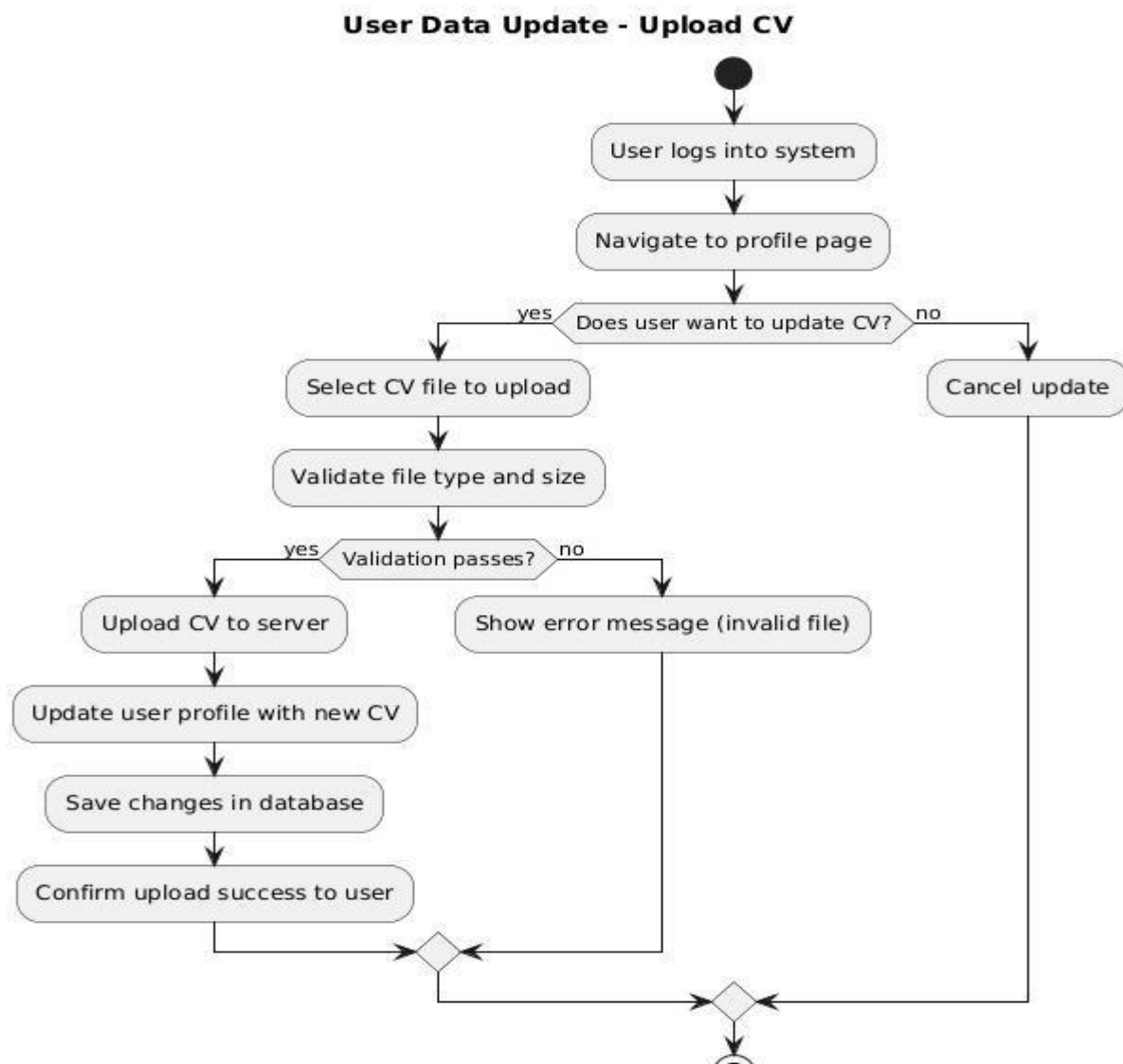


Figure 2-12 Update CV (Activity Diagram)

2.3.5 State Diagram

2.3.5.1 Login State Diagram

As shown in Figure 2-13, this state diagram illustrates the login and password recovery states in the AI Career Coach system. The process starts when the user enters their email, and the system checks whether the email exists in the database. If the email is valid, the user proceeds to enter the password; otherwise, a registration prompt is displayed. In case of invalid credentials, an error message is shown. The diagram also covers the “forgot password” flow, where an OTP is sent to the user’s email to reset the password. After updating the password, the user can retry login and successfully access the system.

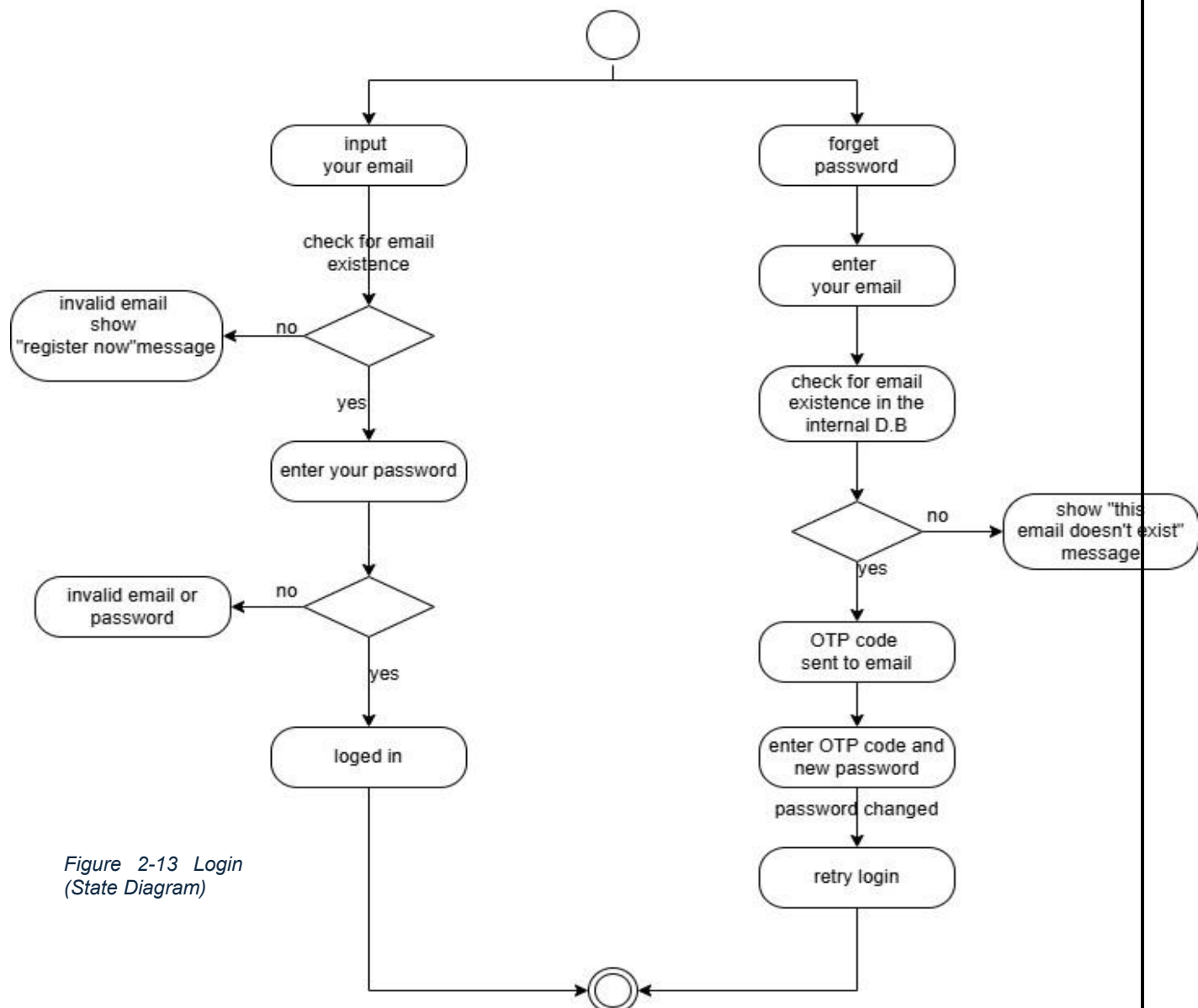


Figure 2-13 Login (State Diagram)

2.3.5.2 Register State Diagram

As shown in Figure 2-14, this state diagram illustrates the user registration process in the AI Career Coach system. The process begins when the user enters their registration data. The system first validates the input format and checks for invalid characters. If the input is invalid, an error message is displayed and the user is asked to retry. A second validation step checks whether the entered data already exists in the system. If the user is new, the data is stored successfully and the registration process is completed.

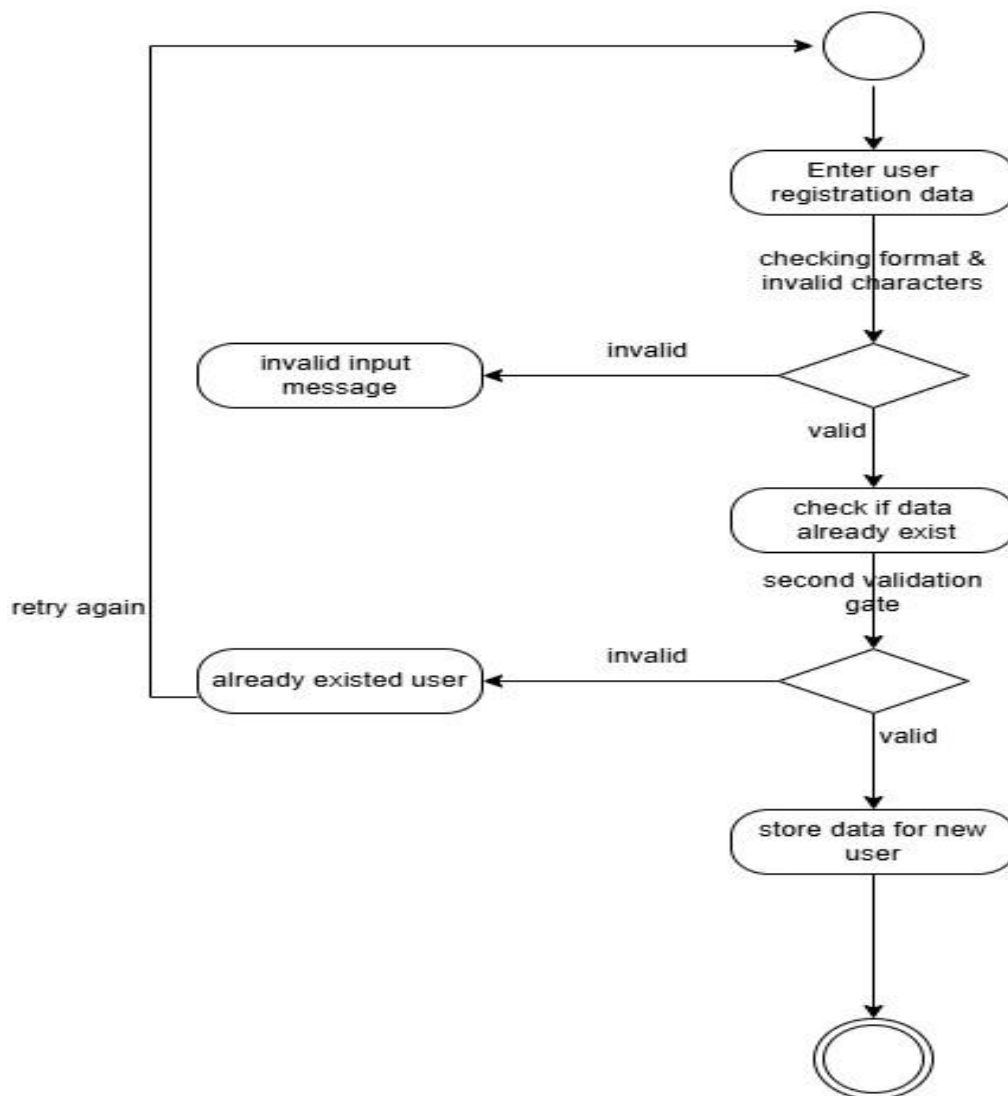


Figure 2-14 Register (State Diagram)

2.3.5.3 Model of AI career coach

As shown in the figure 2-15, this state diagram represents the main operational workflow of the AI Career Coach system. The process begins with the user uploading a CV, which is validated for format and type. Upon successful validation, the system extracts text and applies NLP feature extraction. Intelligent modules including Job Matching, Skill Gap Analysis, Resume Optimization, and Learning Recommendation Engine then process the extracted data. Finally, the system outputs analysis results, optimized CV, job paths to the user.

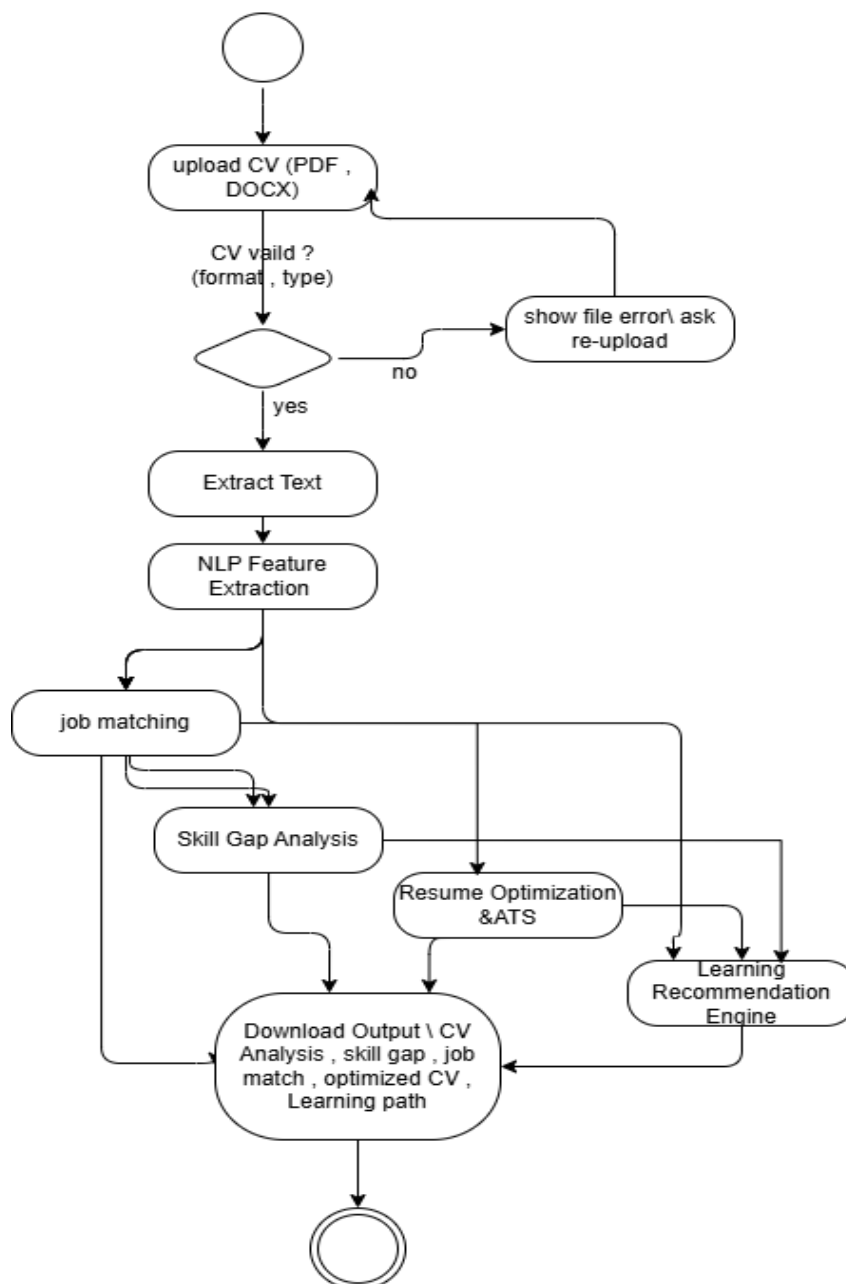


Figure 2-15 System (State Diagram)

2.3.6 Class Diagram

As shown in Figure 2-16, this class diagram represents the main structural components of the AI Career Coach system and their relationships. The *User* class handles user authentication and CV-related actions, while the *System_Controller* manages user data, CV processing, and system operations. Intelligent modules such as *Job_Match*, *Skill_Detector*, *Recommender*, and *ATS_Optimize* are responsible for job matching, skill gap analysis, personalized recommendations, and resume optimization. The *CV_Report* class aggregates analysis results, including matched jobs, skill gaps, and recommended courses. These components interact to generate a comprehensive career analysis and optimized CV report for the user.

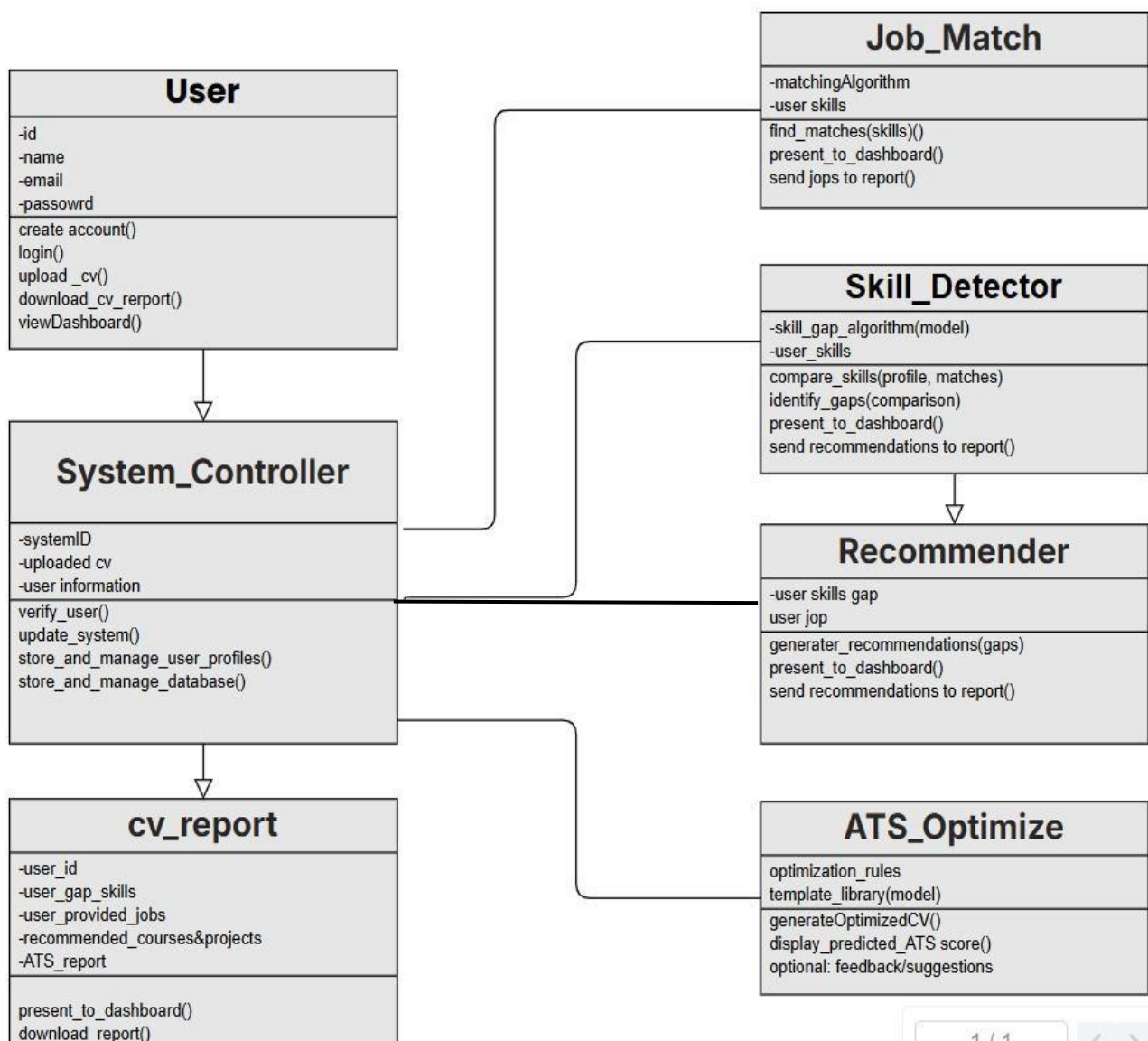


Figure 2-16 Class Diagram

2.3.7 Sequence Diagram

2.3.7.1 Login

As shown in **Figure 2-17**, the login validation sequence diagram illustrates the interaction between the user, frontend interface, backend API, authentication service, and user database.

The diagram demonstrates how user credentials are submitted and verified against stored records.

It also shows the decision flow for successful and failed authentication attempts. Finally, the diagram highlights how the system responds by either granting access to the dashboard or displaying an error message

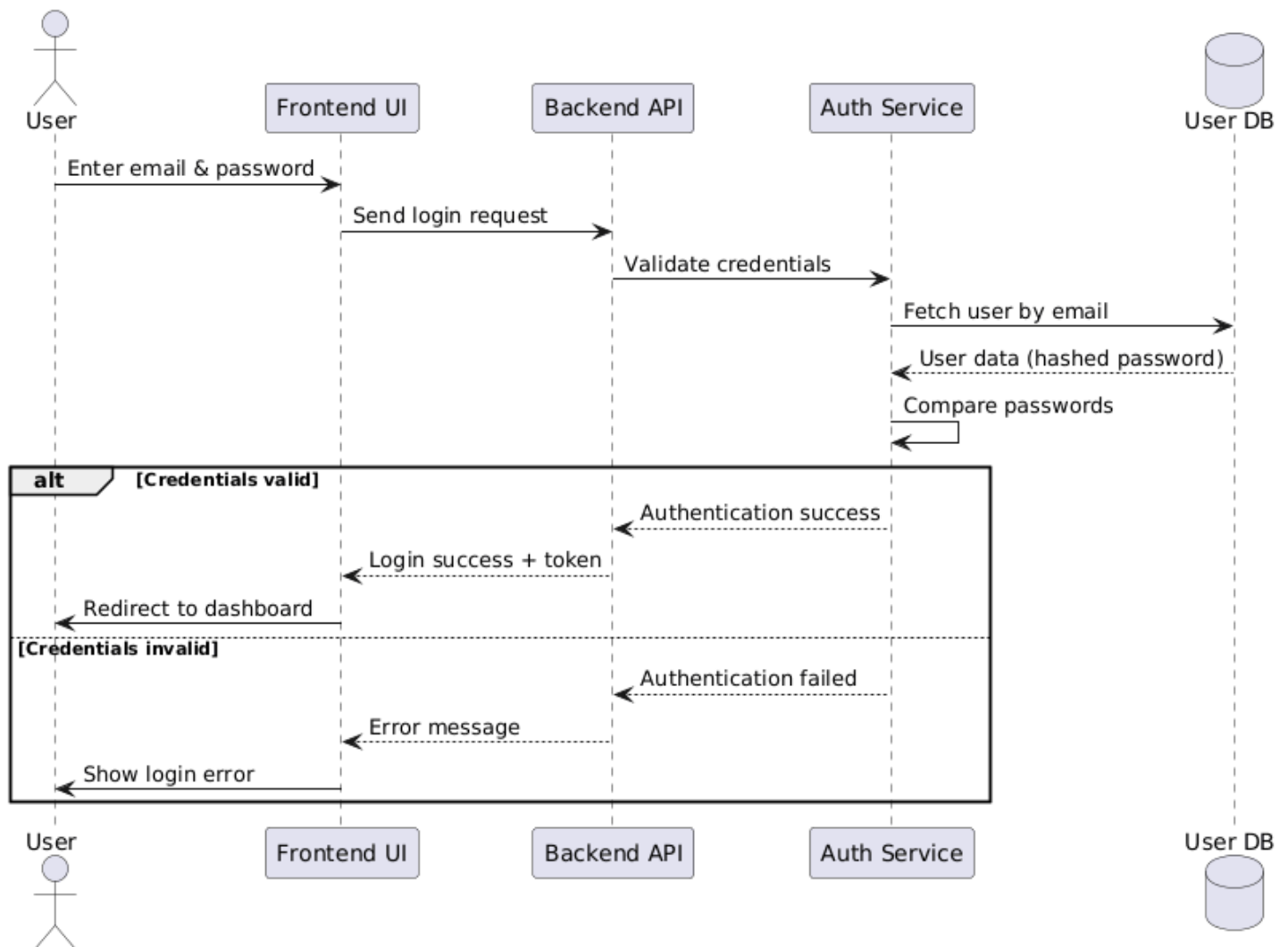


Figure 2-17 Login (Sequence Diagram)

2.3.7.2 Upload CV

As shown in Figure 2-18, the CV upload validation sequence diagram represents the process of verifying uploaded résumé files before further processing.

The diagram illustrates validation of both file type and file size according to predefined constraints. It clearly depicts alternative flows for invalid file formats and oversized files.

Upon successful validation, the system stores the CV and allows the user to proceed to resume analysis.

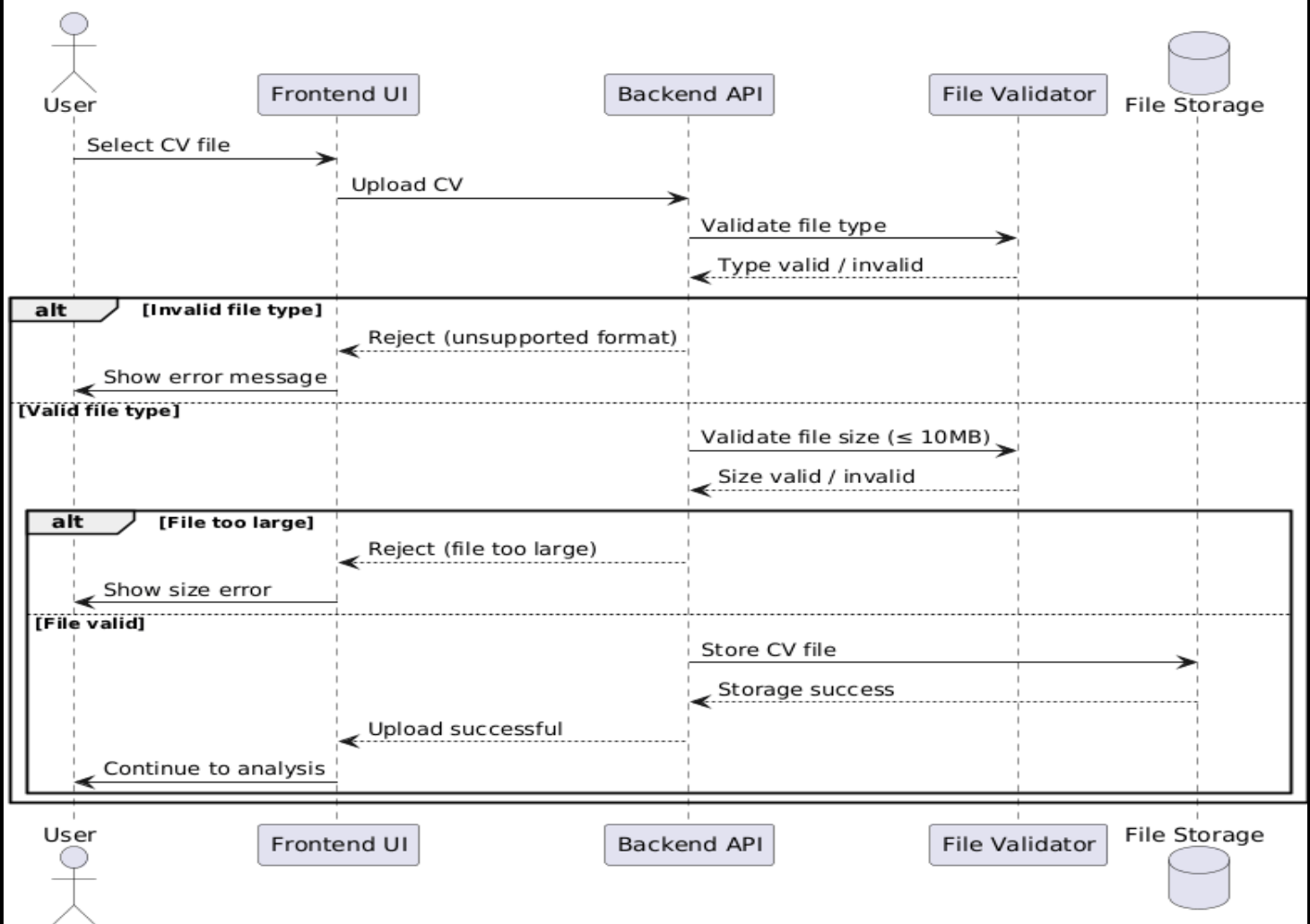


Figure 2-18 Upload CV (Sequence Diagram)

2.3.7.3 System

As shown in figure 2-19, the sequence diagram illustrates the end-to-end workflow of the AI Career Coach system from CV upload to result presentation. The diagram shows how the backend coordinates multiple AI components, including CV analysis, job matching, skill gap detection, resume optimization, and recommendation generation.

It highlights the sequential interaction between the web application, backend API, and intelligent processing engines.

Finally, the processed results are returned to the user through the dashboard, providing job matches, optimized resumes, and personalized learning paths.

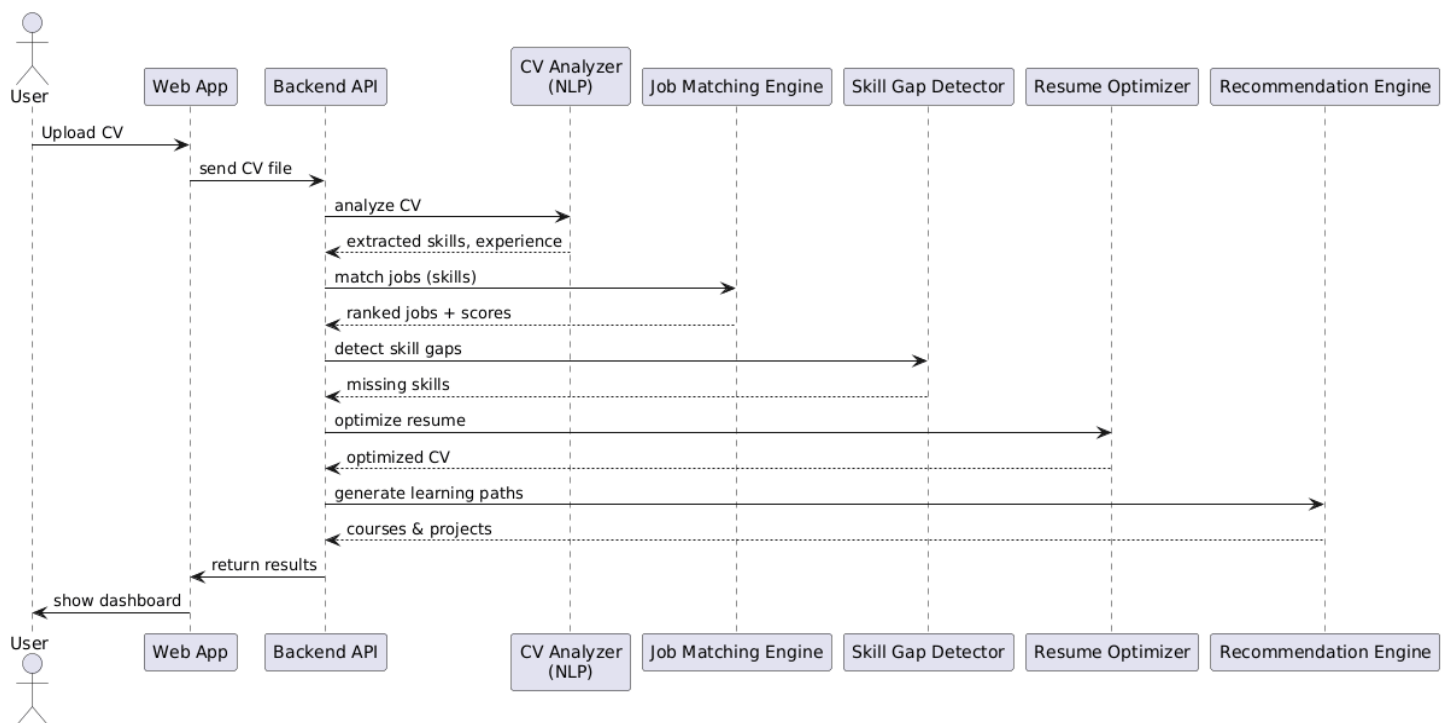
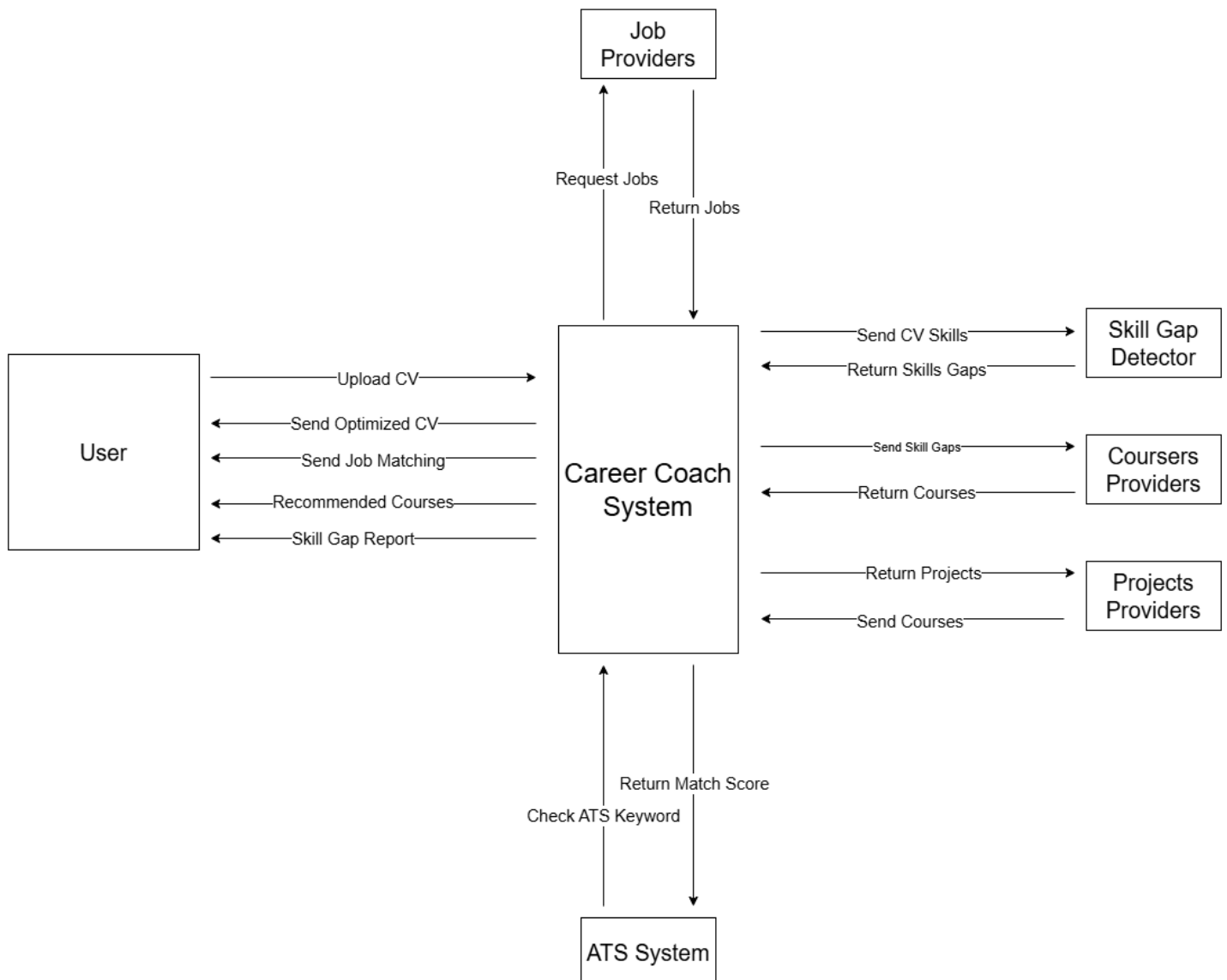


Figure 2-19 System (Sequence Diagram)

2.3.8 Context Diagram

As shown in Figure 2-20, this diagram illustrates the high-level data flow between the AI Career Coach system and external entities. The user uploads a CV to the system and receives optimized CVs, job matching results, recommended courses, and skill gap reports. The system interacts with job providers to request and retrieve available job opportunities. It also communicates with the Skill Gap Detector, course providers, and project providers to analyze skills and generate personalized recommendations. Additionally, the system integrates with an ATS system to evaluate keyword matching and return an ATS score. These interactions enable comprehensive, data-driven career guidance for the user.



2-20 Context Diagram

2.3.9 Data Flow Diagram

2.3.9.1 DFD Level 1

As shown in Figure 2-21, this DFD 1 illustrates the detailed data flow within the AI Career Coach system. The process starts when the user uploads a CV, which is validated and stored in the file storage warehouse. The CV analyzer extracts user features and sends them to the skill gap detection and job provider systems. Identified skill gaps and job offers are processed and stored as reports. Based on the detected gaps, course and project providers return personalized learning recommendations. Additionally, the system checks ATS compatibility to generate an overall career analysis report, which is finally delivered to the user.

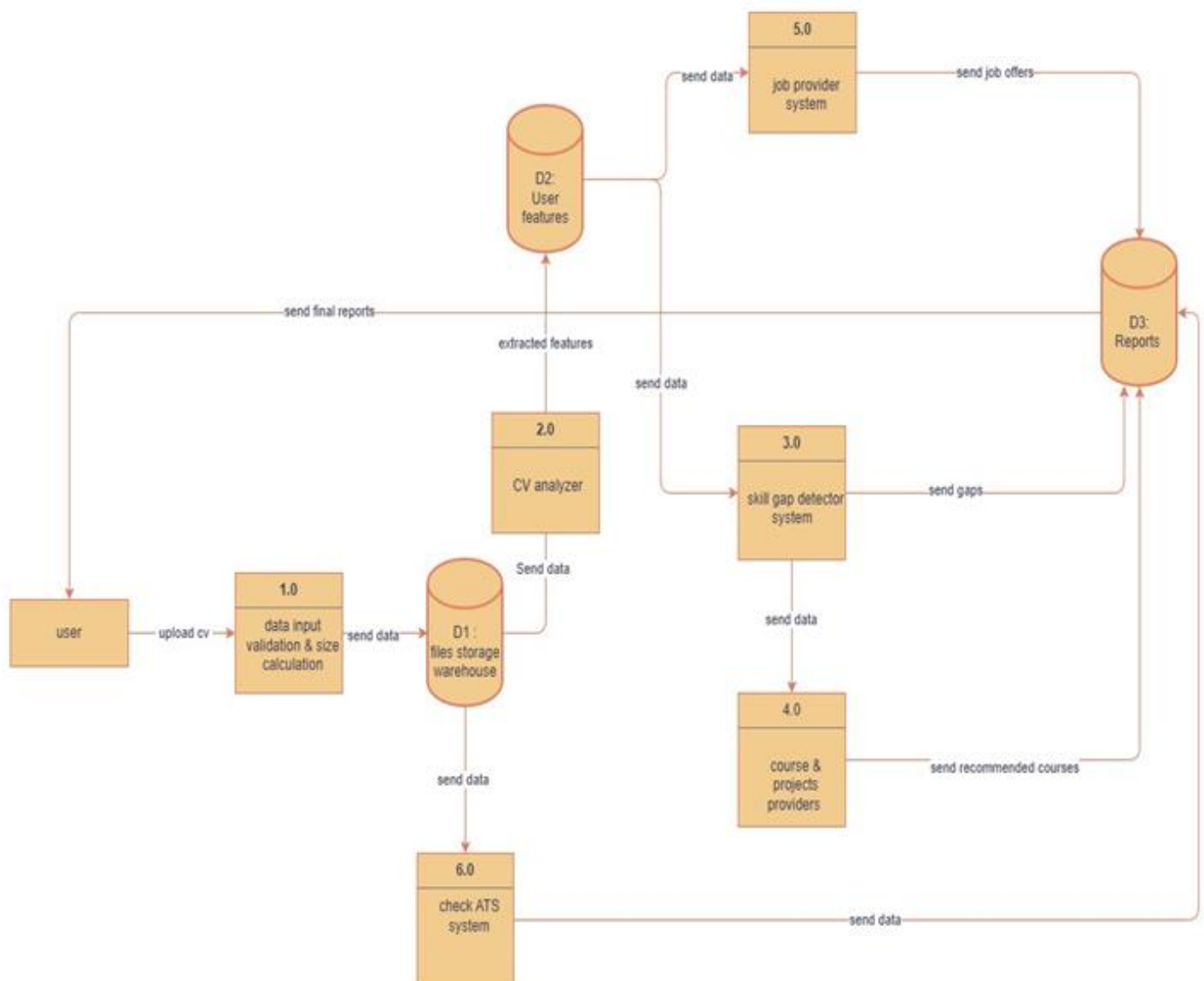


Figure 2-21 DFD L-1

2.3.9.2 DFD Level 2

As shown in **Figure 2-22**, this DFD Level 2 illustrates the detailed internal processes of the AI Career Coach system. The workflow starts with uploading the user's CV, which is validated, parsed, and normalized before being stored in the file storage warehouse. AI modules then extract skills, education, and experience to build user feature profiles. These features are used for job matching, skill gap analysis, and ATS score evaluation. Based on identified skill gaps, suitable courses and projects are recommended. Finally, all results are compiled into reports and delivered to the user.

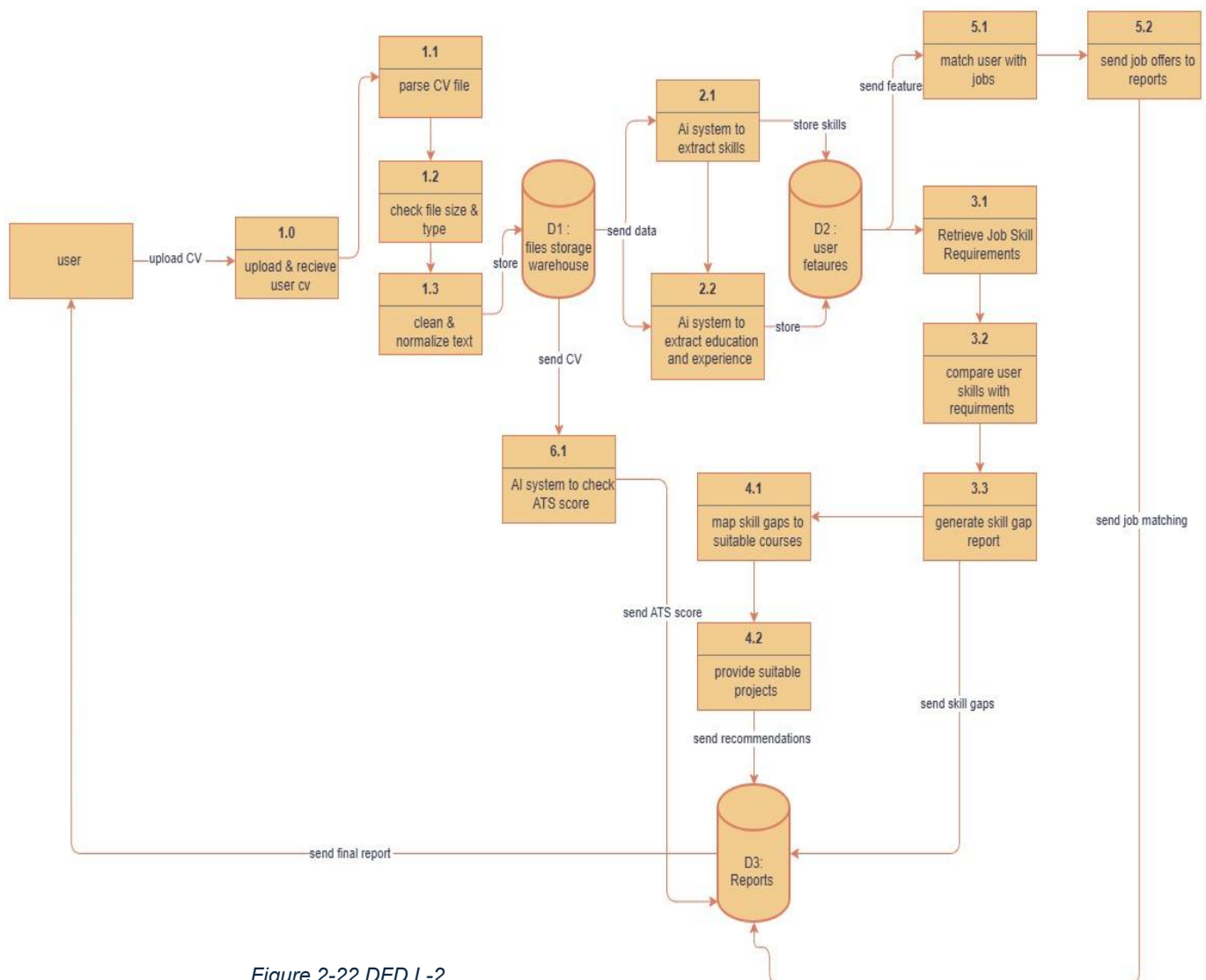


Figure 2-22 DFD L-2

2.3.10 Database Design

2.3.10.1 Schema Diagram

As shown in Figure 2-23, this schema diagram represents the database design of the AI Career Coach system. The User table stores personal and career-related information, while the Resume table manages uploaded CV files and extracted text. Skills are organized through the Skill, User_Skill, and Job_Skill tables to enable accurate skill matching. The Job and Match_Result tables store job details and matching scores for each user. Skill gaps are tracked using the Skill_Gap table, and personalized learning materials are maintained in the Learning_Resource table. This structure supports efficient career analysis, job matching, and recommendation generation.

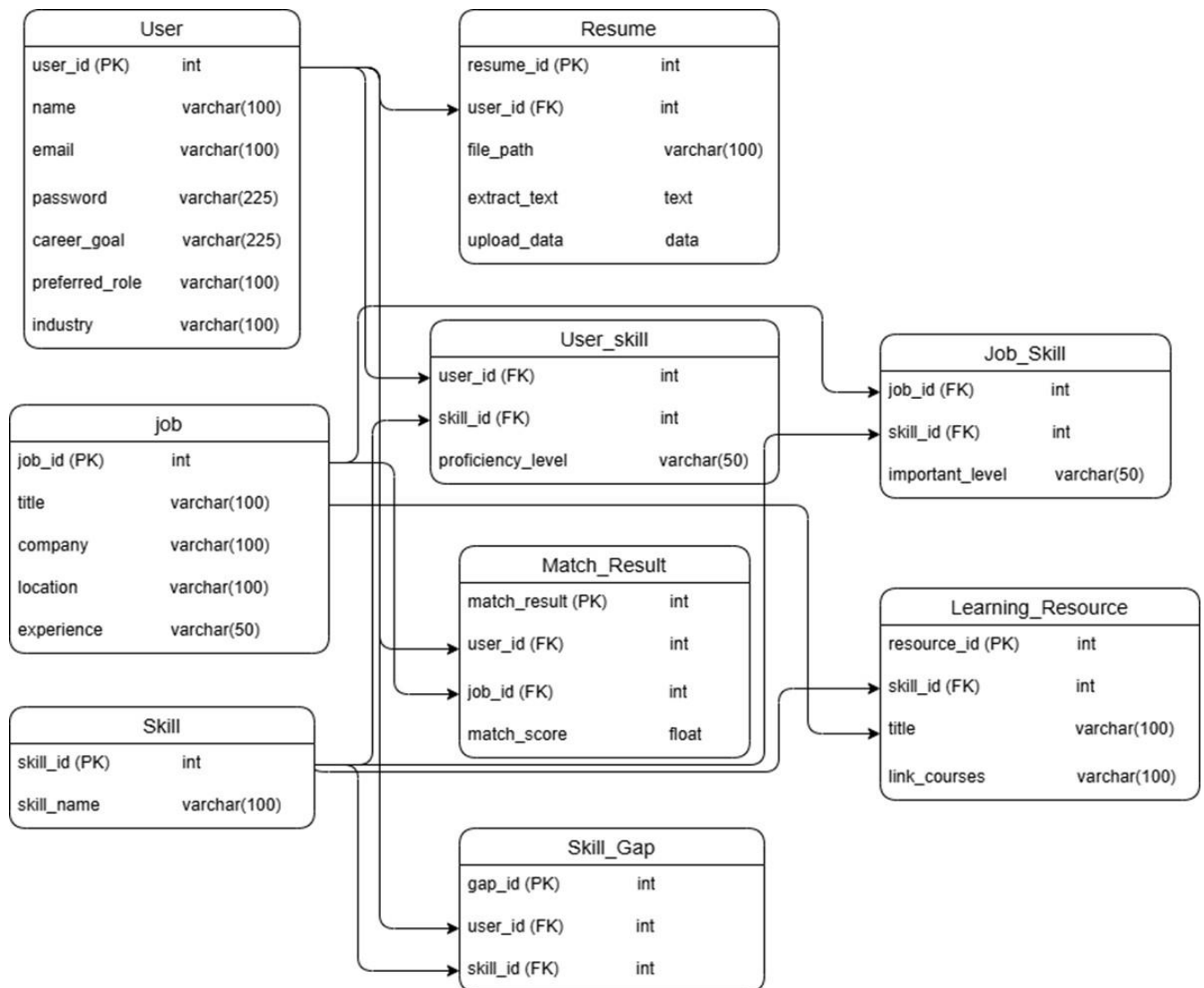


Figure 2-23 Schema Diagram

2.3.10.2 ERD Diagram

As shown in Figure 2-24, this ERD illustrates the main entities and relationships within the AI Career Coach system. The User entity stores personal and career information and is connected to the Resume entity through the upload relationship. Users are matched with jobs via the Matches relationship, which includes a matching score. Skills are linked to users and jobs to represent proficiency levels and required skills. Skill gaps are identified through the relationship between users and skills. The Learning_Resource entity provides courses and materials associated with specific skills to support user development.

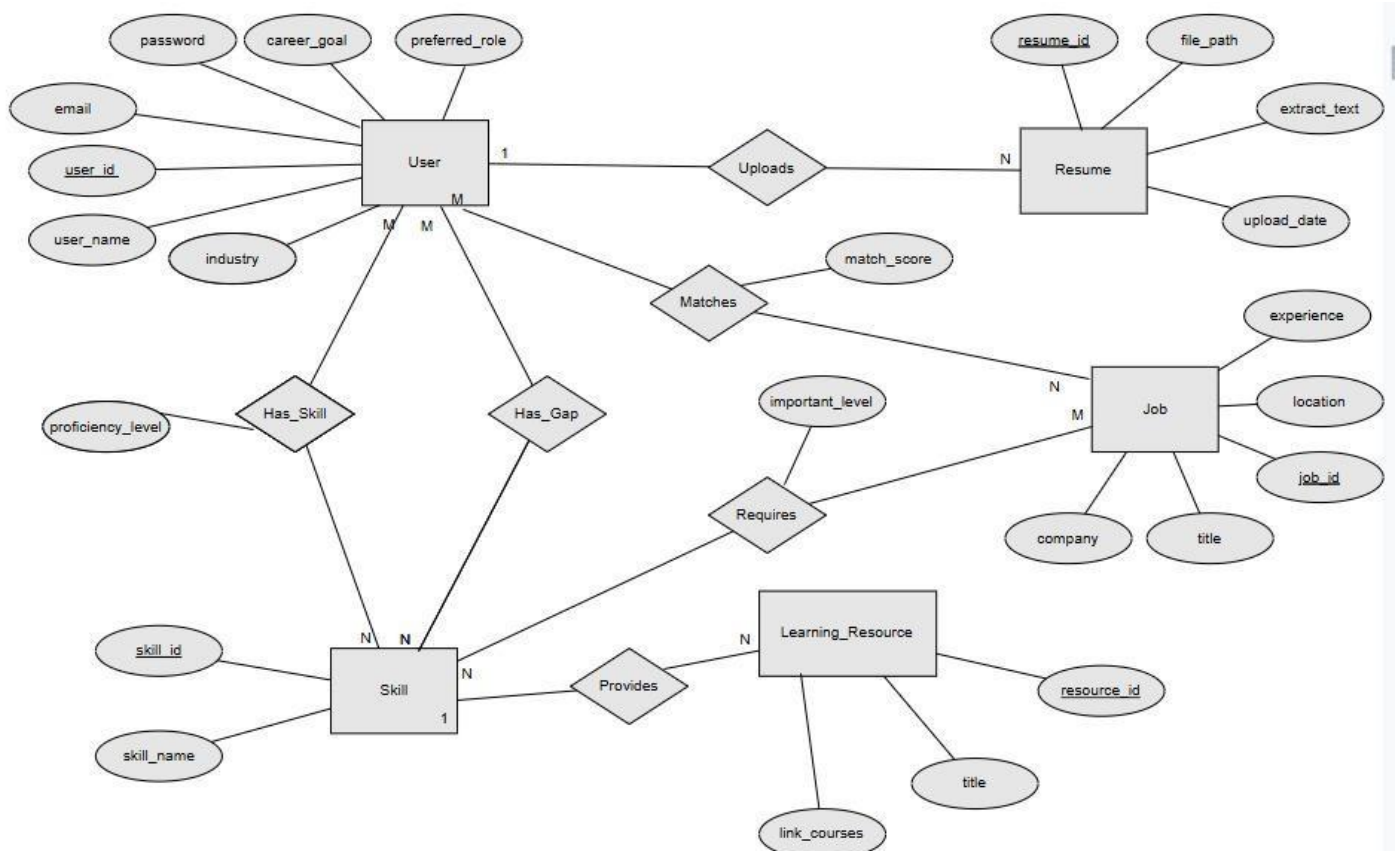


Figure 2-24 ERD Diagram

2.3.11 Mapping Diagram

As shown in Figure 2-25 database are related through primary and foreign keys. The User entity is linked to the Resume entity to associate each user with their uploaded CV. Skills are mapped to users and jobs through the User_Skill and Job_Skill tables to represent proficiency and importance levels. Job matching results are stored in the Match_Result table using user and job identifiers. Skill gaps are tracked by mapping users to missing skills in the Skill_Gap table. This mapping ensures data consistency and supports accurate career analysis and recommendations.

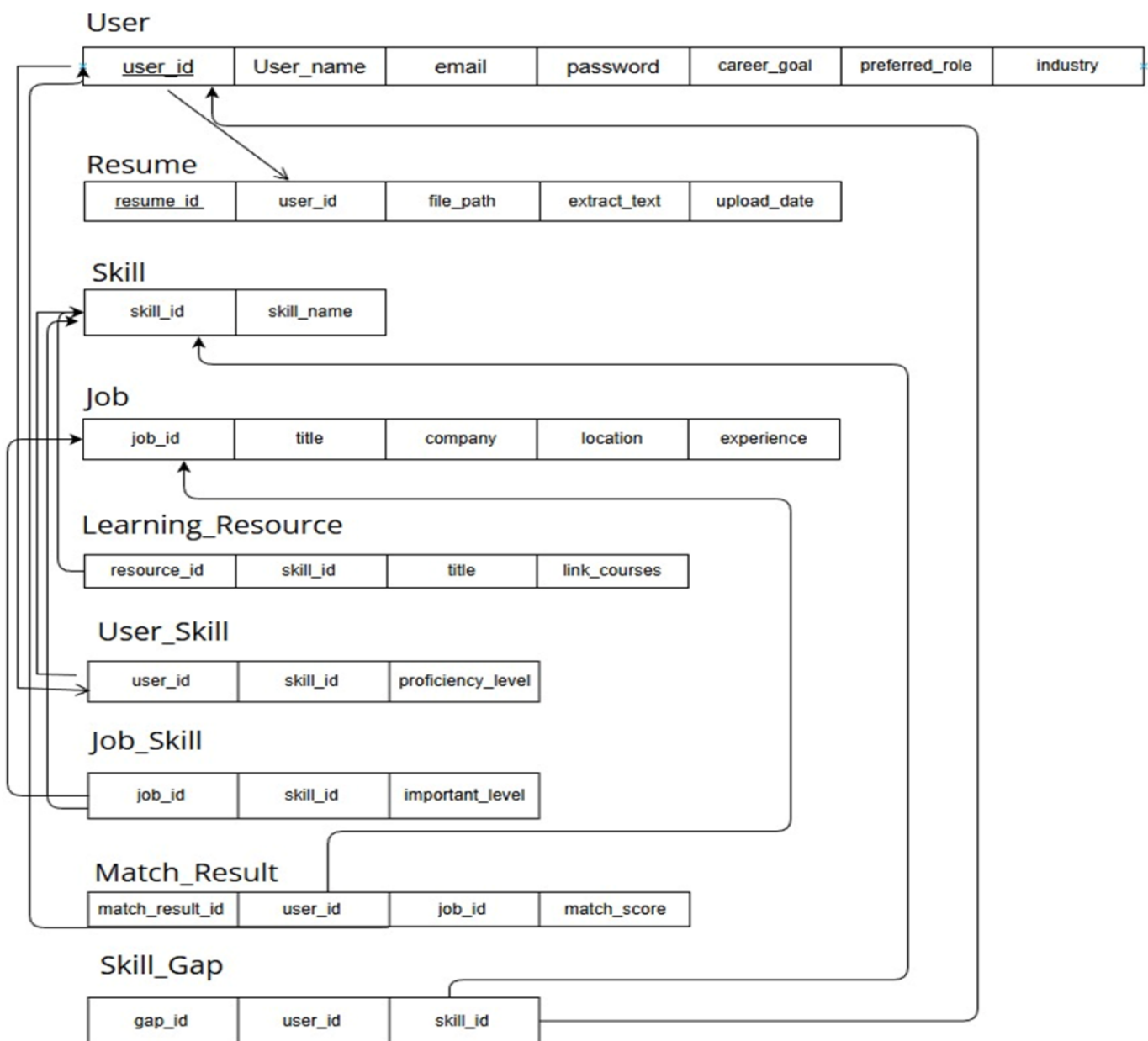


Figure 2-25 Mapping

2.3.12 METHODOLOGY

2.3.12.1 System Units:

The AI Career Coach system is composed of multiple integrated units, where each unit is responsible for a specific functionality within the platform. These units collaborate to provide an intelligent, automated, and user-centered career guidance solution that supports job seekers throughout their career development journey.

The User Management Unit is responsible for handling all user-related operations within the system. It manages user registration, authentication, profile creation, and account maintenance. This unit stores essential user information such as career goals, preferred job roles, and target industries, enabling the system to deliver personalized analysis and recommendations.

The Resume Processing Unit manages the uploading and preprocessing of résumés. It supports multiple document formats, including PDF and DOCX, and performs accurate text extraction. The processed content is structured and prepared for further analysis by downstream components of the system.

The NLP and Information Extraction Unit applies Natural Language Processing techniques to analyze résumé content. It extracts key entities such as skills, education, work experience, achievements, and relevant keywords. The extracted information is normalized and stored in a structured format to support matching, scoring, and analysis tasks.

The Job Matching Unit compares the extracted résumé data with job descriptions obtained from external job platforms or internal databases. It evaluates skill relevance, experience alignment, and keyword similarity to generate an objective Match Score. Based on this score, the unit produces a ranked list of suitable job opportunities for the user.

The Skills Gap Analysis Unit identifies missing, weak, or outdated skills by comparing the user's profile against job requirements. This unit provides clear and actionable insights into areas that require improvement, serving as a foundation for personalized learning recommendations.

The Resume Optimization Unit focuses on improving résumé quality and ATS compatibility. It suggests enhancements to résumé structure, formatting, and keyword usage. Additionally, it uses AI techniques to rewrite poorly phrased sentences and generate an optimized résumé version that users can download and use for job applications.

The Learning and Recommendation Unit delivers personalized learning paths based on identified skill gaps. It recommends relevant online courses, certifications, and practical projects, including real-world GitHub repositories, to help users strengthen their skills and improve employability.

The Dashboard and Reporting Unit provides a centralized interface for visualizing user progress. It displays Match Scores, identified skill gaps, résumé improvement suggestions, and learning recommendations. The unit also supports the generation of downloadable reports in formats such as PDF and Excel.

2.3.12.2 Project Development Methodology (Agile Approach)

The development of the AI Career Coach — Intelligent Career Guidance Platform follows the Agile software development methodology. Agile was selected due to its flexibility, iterative nature, and ability to continuously incorporate user feedback, which is essential for intelligent, user-centered systems.

Although Agile promotes incremental delivery, it maintains the fundamental stages of software development. In this project, these stages are applied as follows:

1. Requirement Definition

In this phase,

the key challenges faced by job seekers were identified, including difficulty in optimizing resumes, understanding required skills for target role, and finding suitable job opportunities and learning paths. Based on these challenges, clear objectives were defined for the platform, such as CV analysis, job matching, skill gap detection, resume optimization, and personalized learning recommendations.

2. System Design

The system architecture was designed using a modular and scalable approach. Core components such as the CV Analyzer, Job Matching Engine, Skill Gap Detector, Smart Resume Optimizer, and Learning Recommendation Module were planned. Agile design principles were applied to allow each module to be developed, tested, and improved independently across iterations.

3. Implementation

The platform was implemented incrementally through short development iterations (sprints). Each sprint focuses on delivering a functional component, such as resume parsing using NLP techniques, job similarity matching using python, Skill Gap Detector and learning recommendation using LangChain or ATS optimization using ML. This iterative process allowed continuous enhancement of system accuracy and performance.

4. Testing and Validation

Testing was performed continuously at the end of each iteration to ensure that the implemented features met the defined requirements. Functional testing, usability testing, and performance evaluation were conducted to verify the reliability, accuracy, and usability of the system for job seekers.

Throughout all stages, the Agile methodology enabled continuous improvement, rapid adaptation to changes, and early identification of issues. The project was supported by modern AI frameworks, NLP tools, and best practices in software engineering to ensure scalability, efficiency, and user satisfaction.

2.3.12.3 Model Selection Justification and Comparison

In the AI Career Coach project, multiple implementation approaches were evaluated for

performing résumé analysis, skill extraction, and job matching. These approaches include building custom Machine Learning or Deep Learning models from scratch, as well as leveraging pre-trained models and APIs provided by established AI platforms such as Hugging Face and external AI services. After careful evaluation, the project adopted a hybrid

approach based on pre-trained models and APIs rather than developing standalone ML/DL models from the ground up.

Use of APIs and Pre-trained Models

APIs and pre-trained models offer a practical and efficient solution for complex Natural Language Processing (NLP) tasks. Platforms such as Hugging Face provide state-of-the-art transformer-based models that have been trained on massive datasets, enabling high accuracy in tasks like semantic similarity, entity extraction, and text classification. By using these models, the system benefits from advanced language understanding without the need for extensive training resources.

Limitations of Building Custom ML/DL Models

Developing Machine Learning or Deep Learning models from scratch requires large labeled datasets, significant computational resources, and extended training and tuning time. In the context of a graduation project with limited time and resources, collecting and annotating high-quality résumé and job description datasets is both challenging and timeconsuming. Additionally, training deep neural networks introduces risks related to overfitting, model instability, and performance inconsistency.

Performance and Accuracy Considerations

Pre-trained transformer models consistently outperform traditional machine learning techniques in NLP tasks due to their contextual understanding of language. These

models capture semantic relationships between skills, job requirements, and experience descriptions more effectively than classical approaches such as TF-IDF or rule-based systems. APIs further optimize inference speed and scalability by handling deployment and optimization internally.

Scalability and Maintainability

Using APIs and pre-trained models significantly improves system scalability and maintainability. Model updates, improvements, and optimizations are handled by the

service providers, allowing the system to evolve with advancements in AI research without requiring major architectural changes. In contrast, maintaining custom ML/DL models would require continuous retraining and monitoring.

Development Efficiency

From a software engineering perspective, integrating APIs and Hugging Face models accelerates development and allows the project to focus on system design, user experience, and intelligent feature integration rather than low-level model training. This approach aligns with real-world industry practices, where AI-powered systems often rely on reliable third-party models and services.

Final Decision

Based on accuracy, resource efficiency, scalability, and development constraints, the AI Career Coach project adopts pre-trained NLP models and AI APIs as the core intelligence layer. This decision ensures robust performance, faster development, and alignment with industry-standard AI deployment practices, while still allowing future enhancement or finetuning if required.

presents a comparative analysis between Large Language Models (LLMs) and custom machine learning/deep learning (ML/DL) models across several technical and operational aspects. The comparison highlights differences in data requirements, accuracy, development time, computational cost, scalability, and maintenance. LLM-based solutions require minimal labeled data and enable faster development due to pre-trained capabilities managed by external providers.

In contrast, custom ML/DL models demand large labeled datasets, longer development cycles, and higher computational resources for training and tuning. The table demonstrates that while LLMs offer high scalability and easier maintenance, custom models provide flexibility at the cost of increased complexity and resource consumption.

Table 2-5 Summary Comparison

Aspect	LLMs Models	Custom ML/DL Models
Data Requirement	No large labeled dataset needed	Requires large-labeled datasets
Accuracy	High	It depends on data and training quality
Development Time	Fast	Long
Computational Cost	Managed by provider	High (training & tuning)
Scalability	High	Limited
Maintenance	Easy	Complex

C h a p t e r Three

3 IMPLEMENTATION

3.1 Introduction

This chapter presents the implementation phase of the AI Career Coach platform and explains how the system was developed using modern web technologies, Artificial Intelligence techniques, and Natural Language Processing (NLP) models. The implementation process focused on building a scalable, modular, and intelligent platform capable of analyzing résumés, identifying skill gaps, matching candidates with suitable job opportunities, and generating personalized learning recommendations.

The system was implemented using a client-server architecture in which the frontend interface communicates with backend APIs responsible for résumé processing, AI analysis, recommendation generation, and database management. The implementation integrates multiple technologies and frameworks to ensure high performance, maintainability, and scalability.

The frontend layer was developed to provide a modern and user-friendly interface that allows users to upload résumés, view analysis results, track skill gaps, and access learning recommendations through an interactive dashboard. The backend layer handles authentication, résumé parsing, NLP pipelines, semantic matching, recommendation generation, and communication with external APIs and databases.

To enhance the intelligence of the platform, the system leverages pre-trained transformer models and external AI APIs for semantic analysis, skill extraction, and résumé-to-job matching. This approach improves accuracy while reducing the complexity and computational cost associated with training deep learning models from scratch.

This chapter describes the implementation of the main system modules, including frontend development, backend APIs, résumé processing pipelines, job matching algorithms, skill gap analysis, recommendation systems, database integration, security mechanisms, and deployment architecture. It also highlights the tools, frameworks, and methodologies used to ensure the successful development of the AI Career Coach platform.

3.1.1 System Architecture

The AI Career Coach system was implemented using a modular architecture to ensure scalability, maintainability, and flexibility. The system consists of multiple interconnected modules including:

- Frontend Interface
- Backend API
- Resume Processing Engine
- NLP & AI Processing Modules
- Job Matching Engine
- Skill Gap Detection Module
- Recommendation Engine
- Database & File Storage

Each module communicates through REST APIs and internal service calls to provide seamless system functionality.

3.1.2 User Interface Implementation (Website)

The AI Career Coach platform provides a modern and interactive user interface designed to simplify résumé analysis and career guidance processes. The interface was implemented using React.js with a responsive dashboard layout to ensure usability, accessibility, and smooth user interaction.

The dashboard integrates multiple intelligent modules including CV analysis, job matching, skill gap detection, résumé optimization, and personalized learning recommendations

3.1.3 CV Neural Analyzer

As shown in Figure 3-1, the CV Neural Analyzer interface represents the initial stage of the AI Career Coach workflow. This module allows users to upload their résumés in supported formats such as PDF and DOCX through an interactive drag-and-drop interface.

The system validates the uploaded file based on:

(File type , File size , Upload integrity)

After successful validation, the résumé is forwarded to the parsing engine where text extraction and preprocessing operations are performed. The interface then allows the user to start the NLP analysis pipeline through the “Analyze Target Document” operation.

This module serves as the primary entry point for résumé analysis and intelligent career processing within the platform

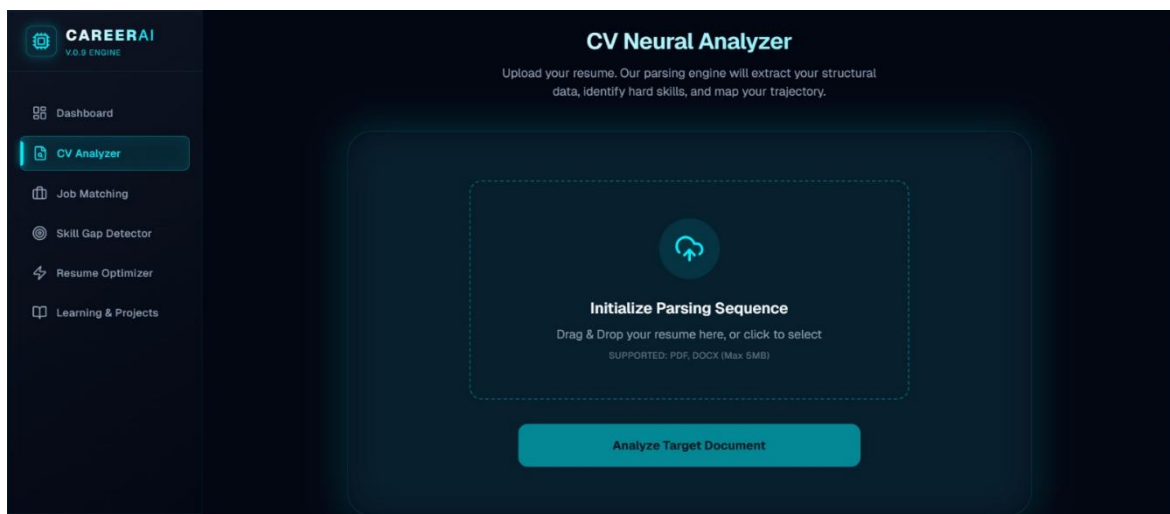


Figure 3-1 CV Upload and Parsing Interface

3.1.4 Resume Analysis Completion Interface

As shown in Figure 3-3, the Resume Analysis Completion interface confirms the successful execution of the résumé parsing and NLP analysis pipeline. After processing the uploaded résumé, the system extracts structural and semantic information such as skills, educational background, work experience, technical keywords, and career-related entities.

The interface then displays the processed document information and provides the user with several interactive operations, including uploading another résumé, viewing the extracted profile data, or running the complete NLP analysis pipeline for deeper processing. In addition, the system generates a unique processing identifier for each uploaded

document to support tracking and analysis management inside the platform.

This stage represents the transition from résumé preprocessing to advanced AI-powered career analysis and recommendation generation.

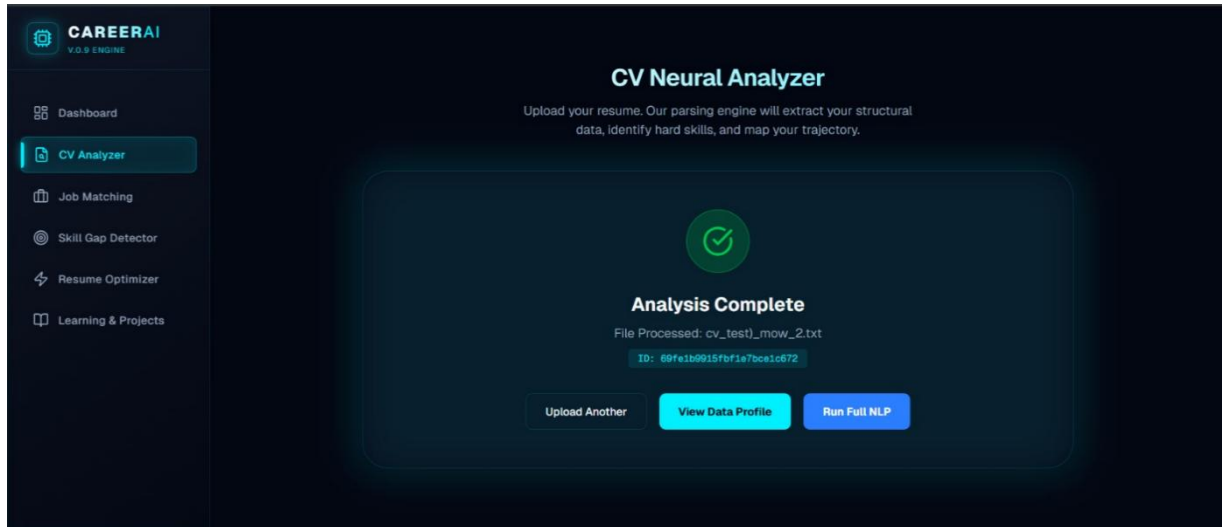


Figure 3-2 Resume Analysis Completion Interface

3.1.5 Skill Gap Detection Interface

As shown in Figure 3-2, the Skill Gap Detection interface presents the missing or underdeveloped competencies identified after analyzing the user's résumé and comparing it with target job requirements.

The system applies semantic analysis and intelligent skill comparison techniques to determine which technologies, tools, and technical competencies are not sufficiently represented in the candidate profile relative to industry expectations. The detected missing skills are displayed through an interactive dashboard to provide users with a clear understanding of the areas that require improvement in order to increase employability and job compatibility.

This module plays an important role in guiding users toward suitable learning paths and personalized recommendations based on their identified skill gaps.

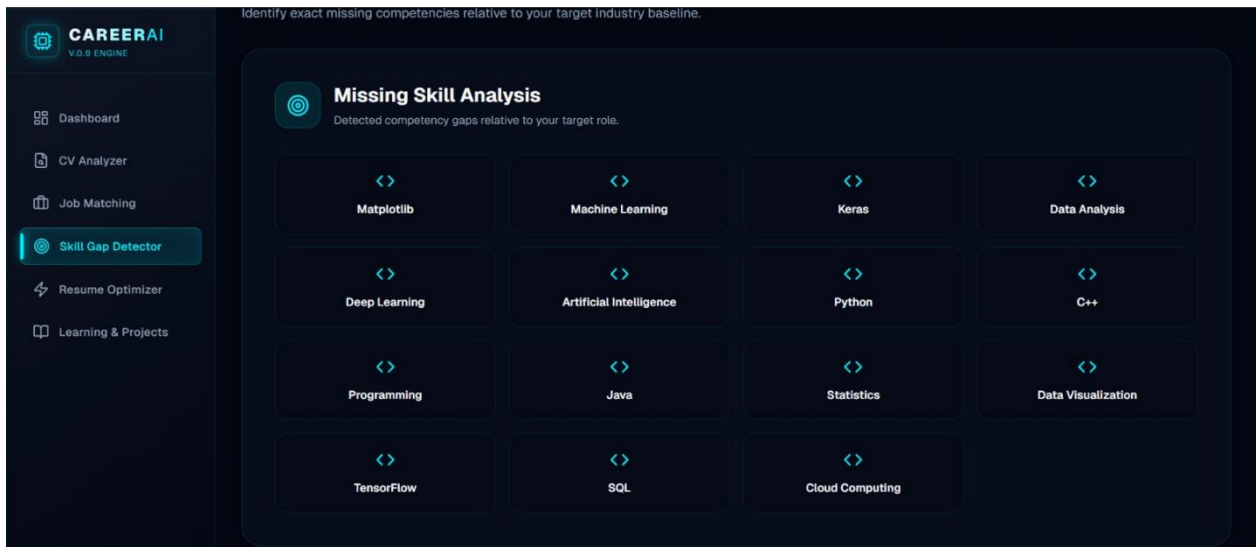


Figure 3-3 Skill Gap Detection Interface

3.1.6 Learning and Projects Recommendation Interface

As shown in Figure 3-4, the Learning and Projects Recommendation interface generates personalized learning resources based on the skill gaps identified during résumé analysis.

The system recommends online courses, technical learning materials, and practical GitHub projects that align with the user's missing competencies and target career path. Recommendations are collected from multiple platforms such as Coursera, Udemy, and GitHub to provide both theoretical learning and hands-on practical experience.

The interface organizes these resources in a structured and user-friendly layout, allowing users to follow guided learning paths that help improve their technical skills and increase job readiness. This module enhances the overall career development process by transforming skill gap analysis into actionable and personalized learning recommendations.

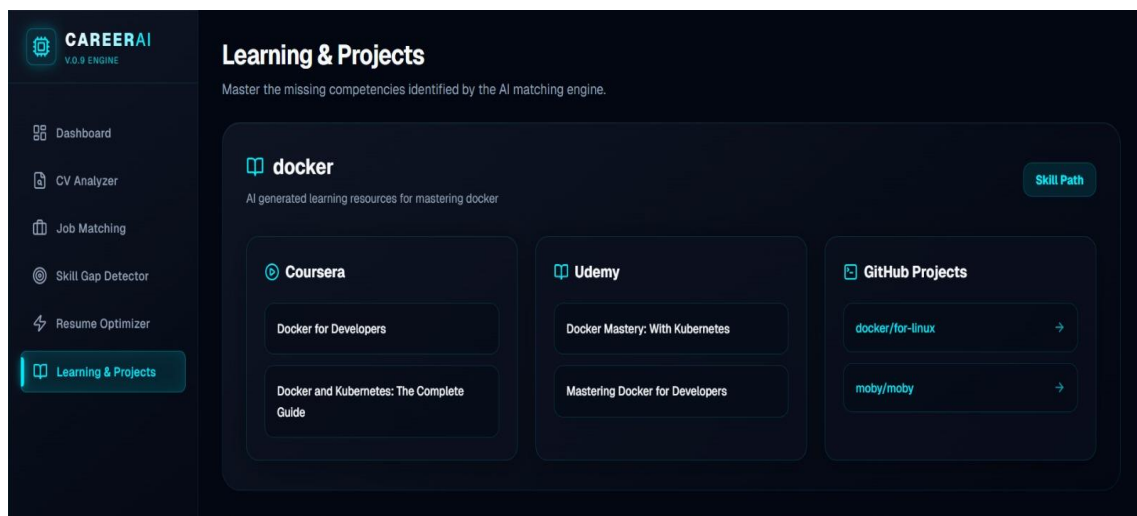


Figure 3-4 Learning and Projects Recommendation Interface

3.1.7 Job Matching Engine Interface

As shown in Figure 3-5, the Job Matching Engine interface presents AI-generated job recommendations based on semantic similarity analysis between the user's résumé and available job descriptions.

The system analyzes extracted skills, professional experience, and technical keywords, then compares them with job requirements using embedding-based similarity techniques and intelligent ranking algorithms. The interface displays detailed job information including job title, company name, location, salary range, and application links, allowing users to directly explore relevant opportunities.

Additionally, the system provides dynamic job refreshing functionality to retrieve updated recommendations and improve matching accuracy according to the user's analyzed profile. This module helps users identify suitable career opportunities that align with their skills and professional background.

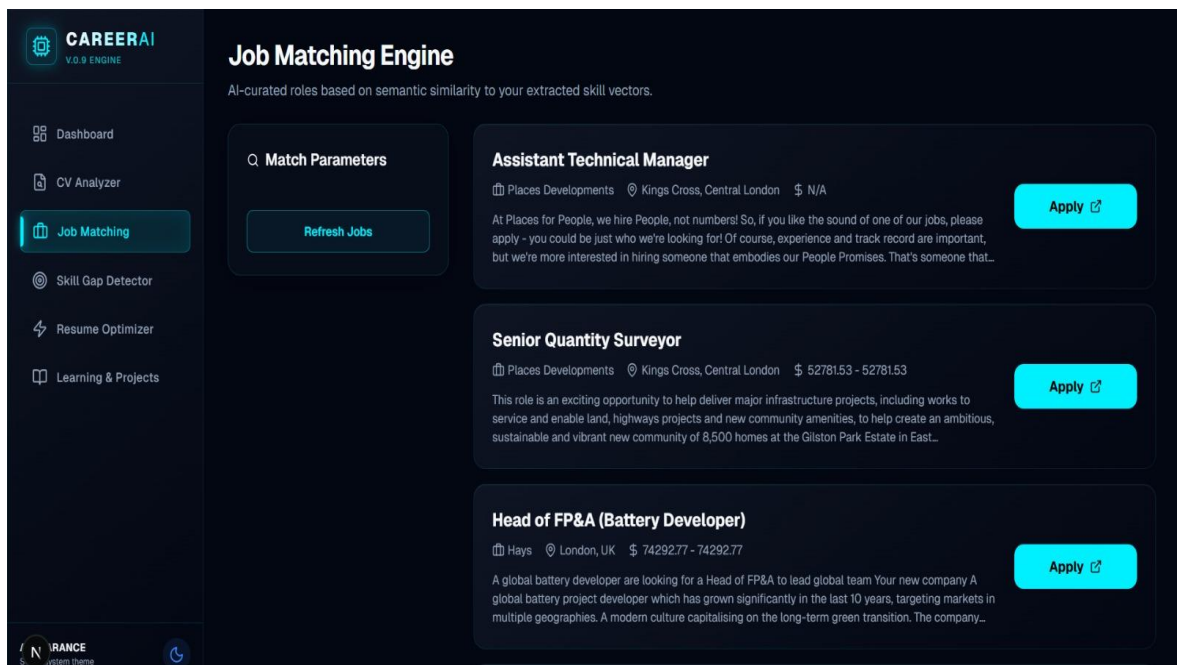


Figure 3-5 Job Matching Engine Interface

3.2 Backend Implementation

The backend of the AI Study Assistant system was implemented using the FastAPI framework due to its high performance, asynchronous capabilities, and ease of integration with modern AI technologies. The backend acts as the core component responsible for handling document processing, embedding generation, vector database communication, retrieval operations, and interaction with Large Language Models (LLMs).

The system follows a modular architecture to improve maintainability, scalability, and code organization. Each module is designed to perform a specific task independently, which simplifies future development and debugging processes.

The backend workflow begins when the user uploads a document through the application interface. The uploaded file is received by the API and passed to the document processing module, where the text is extracted, cleaned, and divided into smaller chunks. These chunks are then converted into vector embeddings using the embedding model and stored inside the vector database for semantic retrieval.

When the user sends a question, the backend converts the query into an embedding vector and searches for the most relevant document chunks inside the vector database. The retrieved context is then combined with the user query and forwarded to the Large Language Model to generate a context-aware response. Finally, the generated answer is returned to the user through the API response.

The backend was developed using asynchronous programming techniques to improve response speed and allow efficient handling of multiple requests simultaneously. FastAPI provides native support for asynchronous endpoints, which enhances the overall system performance, especially during AI model communication and database operations.

The project structure was divided into multiple components to separate responsibilities clearly. The main modules of the backend include:

- **API Routes Module** : Responsible for handling HTTP requests and routing operations.

- Document Processing Module: Responsible for extracting and preprocessing document text.
- Embedding Module: Generates vector embeddings for both document chunks and user queries.
- Vector Database Module: Handles storing and retrieving embeddings using semantic similarity search.
- Generation Module: Communicates with the Large Language Model to generate intelligent responses.
- Configuration Module: Stores environment variables, API keys, and model configurations.

FastAPI automatic documentation features such as Swagger UI were also utilized during development and testing, which simplified API testing and endpoint verification.

The backend architecture was designed to be flexible and extensible, allowing future integration of additional AI models, databases, or advanced Retrieval-Augmented Generation (RAG) techniques without major modifications to the system.

3.2.1 Document Processing Pipeline

The Document Processing Pipeline is responsible for handling uploaded files and preparing their content for semantic analysis and AI processing. This module was implemented to support multiple document formats while maintaining efficient preprocessing and chunk management operations.

The processing workflow begins when the user uploads a document to the system. The backend first validates the uploaded file and determines its extension in order to select the appropriate document loader dynamically. The implementation supports both TXT and PDF document formats using LangChain document loaders.

TXT files are processed using the TextLoader module, while PDF documents are handled using the PyMuPDFLoader module. This design

provides flexibility and allows the system to process different document types using a unified workflow.

The following code snippet demonstrates the dynamic file loader selection process :

```
def get_file_loader(self, file_id: str):  
    file_ext = self.get_file_extension(file_id=file_id)  
    file_path = os.path.join(  
        self.project_path,  
        file_id  
    )  
  
    if not os.path.exists(file_path):  
        return None  
  
    if file_ext == ProcessingEnum.TXT.value:  
        return TextLoader(file_path, encoding="utf-8")  
  
    if file_ext == ProcessingEnum.PDF.value:  
        return PyMuPDFLoader(file_path)  
  
    return None
```

Figure 3-6 Get File Loader

After loading the document, the system extracts both textual content and metadata information such as page references. The extracted content is then passed to the preprocessing and chunking stage.

To improve semantic retrieval quality and contextual understanding, the system divides the extracted text into smaller overlapping chunks using the RecursiveCharacterTextSplitter module from LangChain. Chunking helps reduce context loss and improves similarity matching performance during semantic search operations.

The implemented chunking configuration uses both chunk size and overlap size parameters to preserve contextual continuity between neighboring text segments.

The following code snippet illustrates the chunking implementation:

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=chunk_size,  
    chunk_overlap=overlap_size,  
    length_function=len,  
)
```

Figure 3-7 RecursiveCharacterTextSplitter

After splitting the document, the system creates structured document chunks while preserving metadata associated with each section of the original document. Maintaining metadata with each chunk improves traceability and allows the system to reference the original source during retrieval and response generation operations.

The generated chunks are then forwarded to the embedding generation module, where semantic vector representations are created and stored inside the vector database for intelligent retrieval and contextual response generation.

This pipeline improves processing efficiency, semantic search accuracy, and contextual relevance while enabling the system to handle large documents effectively within the AI workflow.

3.2.2 Embedding and Semantic Representation

The Embedding and Semantic Representation module is responsible for converting textual content into dense numerical vector representations that can be understood and processed by AI systems. This module plays a critical role in enabling semantic understanding, contextual similarity analysis, and intelligent information retrieval within the platform.

After document preprocessing and chunk generation, each text chunk is transformed into a high-dimensional embedding vector using the Cohere multilingual embedding model `embed-multilingual-v3.0`. Unlike traditional keyword-based approaches, embeddings capture semantic meaning and contextual relationships between words, sentences, and technical concepts.

The embedding generation process allows the system to identify semantically related content even when different wording or writing styles are used. This improves the quality of semantic search, document retrieval, and AI-generated responses.

The embedding backend configuration was implemented as follows:

```
EMBEDDING_BACKEND = "COHERE"  
EMBEDDING_MODEL_ID="embed-multilingual-v3.0"
```

Figure 3-8 embedding backend configuration

The multilingual embedding model was selected because of its strong contextual understanding capabilities and support for multiple languages and document structures. This flexibility improves the system's ability to process different types of educational and technical content efficiently.

After generating embeddings, the resulting vectors are stored inside the vector database together with their corresponding metadata and document references. These vectors are later used during semantic similarity search operations to retrieve the most relevant document chunks related to the user query.

The embedding generation workflow can be summarized as follows:

```
chunks = process_file_content(file_content)
embeddings = embedding_model.embed_documents(chunks)
vector_db.store(embeddings)
```

Figure 3-9 embedding generation workflow

Semantic representation significantly improves contextual retrieval accuracy because the system compares vector similarity instead of relying only on exact keyword matching. This enables the platform to understand conceptual relationships between topics, technical terminology, and user questions more effectively.

The embedding module also improves downstream AI tasks including:

- Semantic document retrieval
- Context-aware response generation
- Intelligent question answering
- Similarity matching
- Knowledge extraction

By integrating semantic embeddings into the processing pipeline, the system achieves more accurate and contextually relevant AI-generated responses while improving the overall quality of user interaction and information retrieval.

3.2.3 Vector Database and Semantic Search

The Vector Database and Semantic Search module is responsible for storing embedding vectors and performing semantic similarity retrieval operations. This module enables the system to efficiently search and retrieve contextually relevant document segments based on the semantic meaning of user queries rather than relying solely on traditional keyword matching techniques.

The system uses Qdrant as the vector database backend due to its high-performance vector indexing capabilities, scalability, and efficient nearest-neighbor similarity search operations. Qdrant allows the platform to store high-dimensional embedding vectors together with their associated metadata and document references.

The vector database configuration was implemented as follows:

```
VECTOR_DB_BACKEND = "QDRANT"
```

Figure 3-10 vector database configuration

After generating embeddings for document chunks, the vectors are inserted into the vector database along with metadata information such as document identifiers, chunk references, and source details. This metadata improves traceability and allows retrieved information to be linked back to its original document source.

When the user submits a query, the system first converts the query into an embedding vector using the same embedding model applied during document processing. The generated query vector is then compared against stored vectors inside the database using semantic similarity

```
query_embedding = embedding_model.embed_query(user_query)

results = vector_db.similarity_search(
    query_embedding,
    top_k=5
)
```

Figure 3-11 semantic retrieval workflow

calculations. The semantic retrieval workflow can be summarized as follows:

The similarity search process retrieves the most contextually relevant document chunks based on vector distance metrics such as cosine similarity. This approach enables the system to identify semantically related content even when exact keywords are not present in the stored documents.

Semantic search significantly improves information retrieval quality because it focuses on contextual meaning rather than literal text matching. As a result, the platform can provide more accurate and relevant information during AI-generated response creation.

The Vector Database module also improves:

- Retrieval speed
- Context relevance
- AI response grounding
- Scalability for large document collections
- Intelligent knowledge retrieval

By integrating vector-based semantic search into the AI pipeline, the system achieves efficient contextual retrieval and improves the overall performance of intelligent question-answering operations.

3.2.4 Large Language Model Integration

The Large Language Model (LLM) Integration module is responsible for generating intelligent and context-aware responses based on processed user queries and retrieved document information. This module represents the core AI generation layer of the system and enables natural language understanding, explanation generation, and interactive question answering.

The system integrates the llama-3.1-8b-instant model through the Groq API backend to provide high-speed inference and efficient response generation. The selected model was chosen because of its strong reasoning capabilities, fast response time, and ability to generate coherent contextual outputs suitable for educational and study-assistant applications.

The LLM backend configuration was implemented as follows:

```
GENERATION_BACKEND = "GROQ"  
GENERATION_MODEL_ID = "llama-3.1-8b-instant"
```

Figure 3-12 LLM backend configuration

After semantic retrieval operations are completed, the retrieved contextual document chunks are combined with the user query and forwarded to the language model. The model then analyzes the provided context and generates a relevant response based on the retrieved information and the user request.

The response generation workflow can be summarized as follows:

```
context = retrieve_relevant_chunks(user_query)  
  
response = llm.generate(  
    query=user_query,  
    context=context  
)
```

Figure 3-13 response generation workflow

The integration between semantic retrieval and the language model improves response accuracy and contextual relevance by grounding generated outputs on retrieved document content. This reduces incorrect or unrelated responses and improves the reliability of AI-generated answers.

The language model is responsible for multiple intelligent tasks داخل النظام including:

- Question answering
- Contextual explanation generation
- Summarization
- Educational assistance
- Intelligent response generation
- Study support interactions

LangChain was used to simplify communication between the application backend, retrieval pipeline, and language model APIs. The framework helps organize prompt management, model invocation, and AI workflow orchestration in a modular and maintainable manner.

The implemented LLM integration provides real-time interactive responses and enhances the overall user experience by enabling natural and context-aware communication between users and the AI-powered study assistant platform.

3.2.5 AI Query Processing Workflow

The AI Query Processing Workflow represents the complete operational pipeline followed by the system to process user requests and generate intelligent responses. The workflow combines document processing, semantic representation, vector retrieval, and Large Language Model generation to provide context-aware answers and educational assistance.

The workflow begins when the user uploads one or more documents through the application interface. The uploaded files are first validated and processed by the document processing module, where textual content is extracted and cleaned before being divided into smaller semantic chunks.

After preprocessing, the generated chunks are converted into embedding vectors using the Cohere embedding model. These vectors are then stored inside the Qdrant vector database together with their corresponding metadata and document references.

When the user submits a query through the chat interface, the system converts the query into an embedding vector using the same embedding model to ensure semantic consistency between stored documents and user input.

The generated query embedding is then compared with stored vectors inside the vector database using semantic similarity search operations. The system retrieves the most contextually relevant document chunks based on vector similarity calculations.

After retrieval, the selected contextual information is combined with the original user query and forwarded to the Large Language Model through the Groq API. The LLM analyzes both the query and the retrieved context to generate an intelligent and context-aware response.

The generated answer is finally returned to the frontend interface and displayed to the user in real time.

The AI query processing workflow can be summarized in the following sequence:

1. User uploads document
2. Text extraction and preprocessing
3. Text chunk generation
4. Embedding vector generation
5. Vector storage in Qdrant
6. User submits query
7. Query embedding generation
8. Semantic similarity retrieval
9. Context forwarding to LLM
10. AI response generation
11. Response displayed to the user

The following simplified code snippet illustrates the query processing workflow:

```
query_embedding = embedding_model.embed_query(user_query)

relevant_chunks = vector_db.similarity_search(
    query_embedding,
    top_k=5
)

response = llm.generate(
    query=user_query,
    context=relevant_chunks
)
```

Figure 3-14 the query processing workflow

This workflow enables the system to generate more accurate and contextually relevant responses by combining semantic retrieval techniques with advanced Large Language Model capabilities. It also improves scalability and allows efficient handling of large document collections and multiple user queries simultaneously.

3.2.6 API Implementation

The API layer of the system was implemented using the FastAPI framework to provide efficient communication between the frontend interface, AI processing modules, vector database, and language model services. The API architecture follows a modular routing structure where endpoints are separated according to their functionality, improving maintainability and simplifying backend development.

The API routes were organized into multiple modules including base routes, data processing routes, and NLP-related routes. Each route module is responsible for handling a specific category of system operations.

The base API router was implemented using FastAPI `APIRouter` objects with versioned endpoints to support scalability and future API expansion.

The following code snippet demonstrates the base API route configuration:

```
base_router = APIRouter(  
    prefix="/api/v1",  
    tags=["api_v1"],  
)
```

Figure 3-15 base API route

The backend provides multiple API endpoints responsible for document upload, preprocessing, AI analysis, semantic search, skill extraction, recommendation generation, and job matching operations.

The document upload endpoint receives files uploaded from the frontend interface and stores them inside the project workspace after validation and preprocessing checks.

The following simplified code snippet illustrates the upload endpoint implementation:

```
@data_router.post("/upload/{project_id}")
async def upload_data(
    request: Request,
    project_id: str,
    file: UploadFile
):
```

Figure 3-16 upload endpoint

After document upload, the processing endpoint performs chunk generation and semantic preprocessing operations before forwarding the processed content to the embedding generation pipeline.

The backend also includes NLP-related endpoints responsible for AI-powered operations such as:

- Skill extraction
- Skill gap analysis
- Learning recommendation generation
- ATS score analysis

The NLP router was implemented as follows:

```
nlp_router = APIRouter(
    prefix="/api/v1/nlp",
    tags=["api_v1"]
)
```

Figure 3-17 NLP router

One of the core AI endpoints is responsible for generating intelligent responses and extracting information from processed documents using the integrated language model.

The following code snippet demonstrates the AI answer generation endpoint:

```
@nlp_router.post("/index/answer/{project_id}")
async def answer_rag(
    request: Request,
    project_id: str
):
```

Figure 3-18 AI answer generation endpoint

The system also integrates external job APIs to retrieve real-world job opportunities dynamically. The backend communicates with external recruitment services and returns structured job recommendations to the frontend dashboard.

The following code snippet demonstrates the job retrieval endpoint:

```
@nlp_router.post("/index/jobs/{project_id}")
async def get_jobs(
    what: str,
    where: str
):
```

Figure 3-19 job retrieval endpoint

The implemented API architecture supports asynchronous request handling to improve performance and reduce response latency during AI processing and database communication operations.

FastAPI automatic documentation tools such as Swagger UI were utilized during development and testing phases to simplify endpoint verification and API interaction testing.

The modular API implementation improves scalability and allows future integration of additional AI services, external APIs, and advanced NLP functionalities without requiring major architectural modifications.

References

1 - OpenAI. (2023). *GPT Models and Embeddings Documentation*.

This documentation is used for natural language understanding, resume improvement, text generation, and explanation generation within the system.
<https://platform.openai.com/docs> <https://platform.openai.com/docs/api-reference/introduction>

2 - LangChain Documentation. (2023). *Building LLM-Powered Applications with LangChain*.

This reference is used for prompt engineering, document loaders, and building RetrievalAugmented Generation (RAG) pipelines.
https://docs.langchain.com/oss/python/integrations/text_embedding/openai

3 - Facebook AI Research (FAISS). *FAISS: A Library for Efficient Similarity Search*. FAISS is used as a vector database to perform fast similarity search and retrieval of resume and job embeddings.

<https://faiss.ai/>
<https://github.com/facebookresearch/faiss>

4 - Applicant Tracking Systems (ATS) Studies – Jobscan & Indeed Engineering Blogs. These studies are referenced for resume keyword optimization, structure improvement, and ensuring ATS compatibility.

<https://www.jobscan.co/blog/> <https://engineering.indeedblog.com/>

5 - GitHub. *GitHub Platform*.

GitHub is used to recommend real-world projects aligned with required technical skills and career paths. <https://github.com>

6 - GitHub REST API Documentation.

This reference is used to access repositories, programming languages, and project metadata for recommendation purposes.
<https://docs.github.com/en/rest>

7 - Coursera. *APIs and Integration Documentation*.

Coursera resources are used to support course recommendations and skill-based learning path generation. <https://www.coursera.org/learn/apis>

8 - Udemy. *Public Course Data and Platform Documentation.*

Udemy content is referenced for identifying relevant online courses and certifications. <https://www.udemy.com>

9 - Hugging Face. *Machine Learning Models and Datasets Platform.*

Hugging Face is used to access pre-trained NLP models and embeddings instead of training deep learning models from scratch.

<https://huggingface.co>

10 - Kaggle. *Datasets and Notebooks Platform.*

Kaggle is used as a source for datasets and example notebooks related to resumes, job +descriptions, and NLP tasks.

<https://www.kaggle.com>

11 - FastAPI Documentation.

FastAPI is used to build high-performance backend APIs for the system.

<https://fastapi.tiangolo.com/>

12 - FastAPI GitHub Repository.

This reference provides implementation details and best practices for FastAPI development. <https://github.com/FastAPI/FastAPI>